Neural Networks and Deep Learning
RNNs, Autoencoders

M. Vazirgiannis

Data Science and Mining Team (DASCIM), LIX
École Polytechnique
http://www.lix.polytechnique.fr/dascim
Google Scholar: https://bit.ly/2rwmvQU
Twitter: @mvazirg

October, 2025

Outline

1. MLP and CNN applications to NLP
   - MLPs for Word embeddings
2. Recurrent NNs + LSTMs
3. Auto-encoders for unsupervised learning

Language model

- Goal: determine $P(s = w_1 \dots w_k)$ in some domain of interest

$$P(s) = \prod_{i=1}^{k} P(w_i | w_1 \dots w_{i-1})$$

e.g. $P(w_1 w_2 w_3) = P(w_1) P(w_2|w_1) P(w_3|w_1 w_2)$

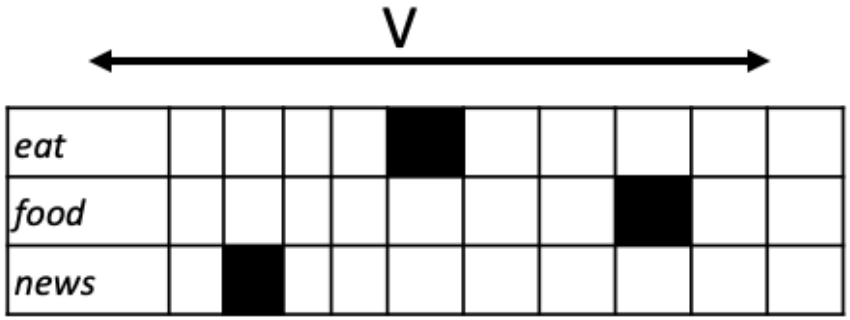
- Traditional n-gram language model assumption:"the probability of a word depends only on context of n-1 previous words"

$$\Rightarrow \hat{P}(s) = \prod_{i=1}^{k} P(w_i | w_{i-n+1} \dots w_{i-1})$$

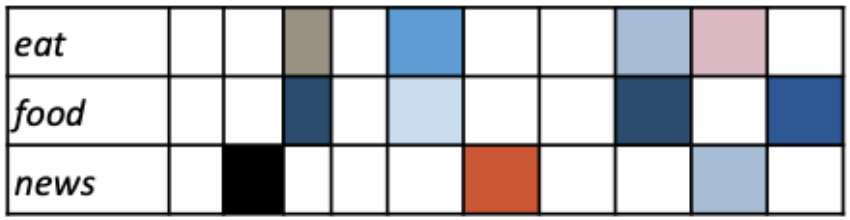
- Typical ML-smoothing learning process (e.g., Katz 1987):
  1. compute $\hat{P}(w_i | w_{i-n+1} \dots w_{i-1}) = \frac{\# w_{i-n+1} \dots w_{i-1} w_i}{\# w_{i-n+1} \dots w_{i-1}}$ on training corpus
  2. smooth to avoid zero probabilities

Representing Words

- One-hot vector
  - high dimensionality
  - sparse vectors
  - dimensions=$|V|$ ($10^6 < |V|$)
  - unable to capture semantic similarity between words



- Distributional vector
  - words that occur in similar contexts, tend to have similar meanings
  - each word vector contains the frequencies of all its neighbors
  - dimensions=$|V|$
  - computational complexity for ML algorithms



Example

- We should assign similar probabilities (discover similarity) to *Obama speaks to the media in Illinois* and the *President addresses the press in Chicago*
- This does not happen because of the "one-hot" vector space representation

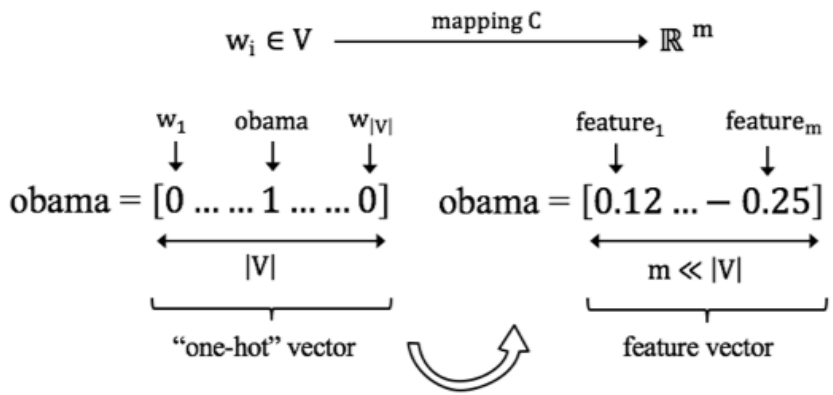
One hot

$$\left.\begin{array}{l} \text{obama} = [0\,0\,0\,0\ \ldots\ 0\,1\,0\,0] \\ \text{president} = [0\,0\,0\,1\ \ldots\ 0\,0\,0\,0] \end{array}\right\} \overrightarrow{\text{obama}}.\overrightarrow{\text{president}} = \vec{0}$$

$$\left.\begin{array}{l} \text{speaks} = [0\,0\,1\,0\ \ldots\ 0\,0\,0\,0] \\ \text{addresses} = [0\,0\,0\,0\ \ldots\ 0\,0\,1\,0] \end{array}\right\} \overrightarrow{\text{speaks}}.\overrightarrow{\text{addresses}} = \vec{0}$$

$$\left.\begin{array}{l} \text{illinois} = [1\,0\,0\,0\ \ldots\ 0\,0\,0\,0] \\ \text{chicago} = [0\,1\,0\,0\ \ldots\ 0\,0\,0\,0] \end{array}\right\} \overrightarrow{\text{illinois}}.\overrightarrow{\text{chicago}} = \vec{0}$$

Word Embedding



Co-occurrence matrix

- I like to fly
- I enjoy learning to fly
- I like learning english

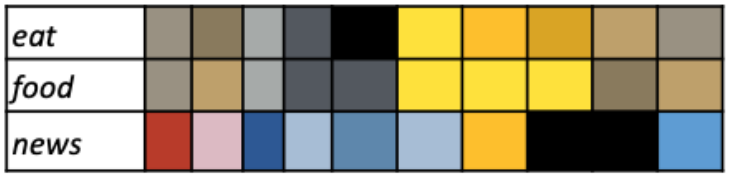
	I	like	fly	enjoy	learning	english
I		1	1	1	1	1
like	1		1		1	1
fly	1			1		
enjoy	1		1		1	
learning	1	1	1			1
english	1	1			1	

- The matrix is large and sparse
  - Storage
  - Classification
- Learn *a low dimensional dense representation of words*

Representing Words

Word embedding

- store the same contextual information in a low-dimensional vector
- **densification** (sparse to dense)
- **compression**
  - dimensionality reduction
  - dimensions=m
    $100 < m < 500$
- able to capture semantic similarity between words
- learned vectors (unsupervised)
- Learning methods
  - SVD
  - word2vec
  - GloVe



SVD word embeddings

- Dimensionality reduction on co-occurrence matrix
- Create a $|V|x|V|$ word co-occurrence matrix X
- Apply SVD $X = USV^T$
- Take first k columns of U
- Use the k-dimensional vectors as representations for each word
- Able to capture semantic and syntactic similarity

Latent Semantic Indexing (LSI) - II

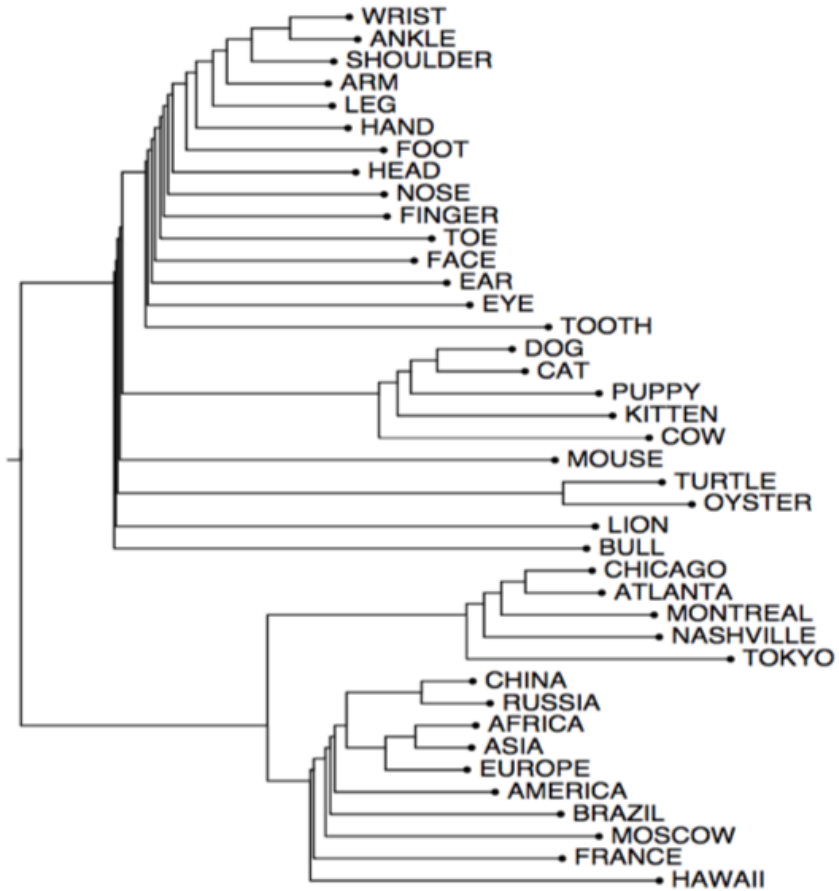
- The initial matrix is SVD decomposed as: $A = ULV^T$
- Choosing the top-k singular values from L we have: $A_k = U_k L_k V_k^T$
- L_k: square $k \times k$ - top-k singular values of the diagonal in matrix L
- U_k: $m \times k$ matrix - first k columns in U (left singular vectors)
- V_k^T: $k \times n$ matrix - first k lines of V^T (right singular vectors)

Typical values for $k \sim 200 - 300$ (empirically chosen based on experiments appearing in the bibliography)

LSI capabilities

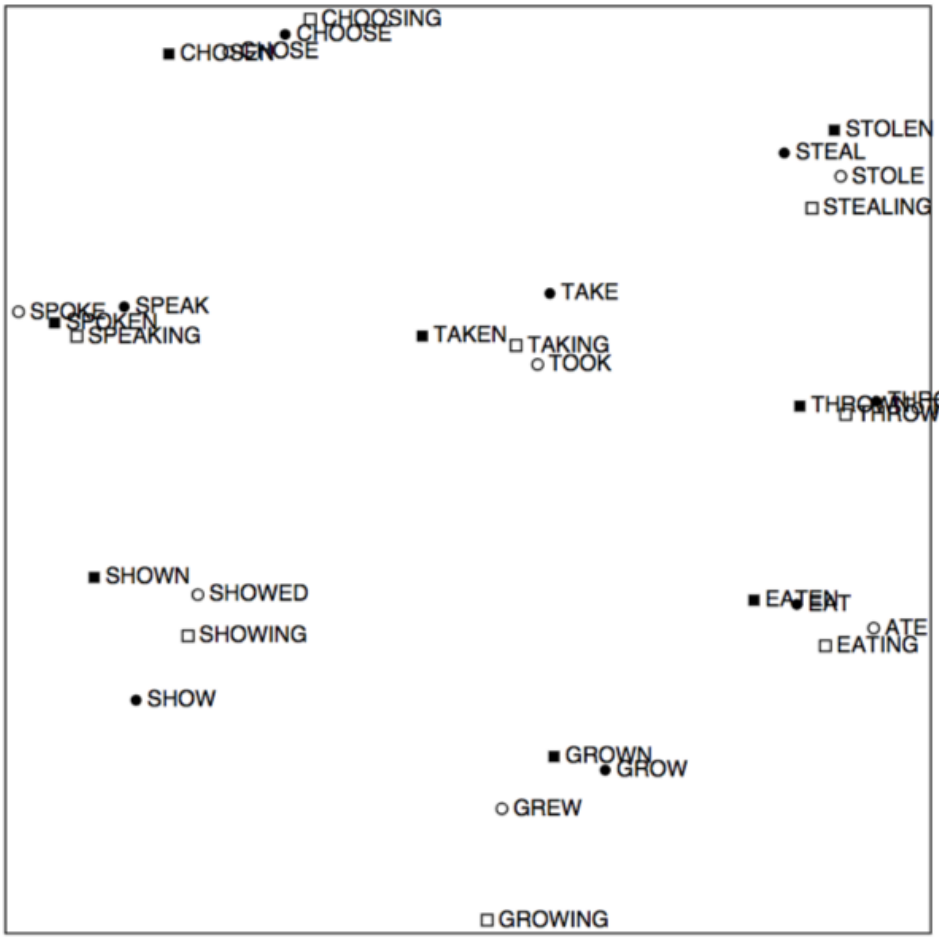
- Term to term similarity: $A_k A_k^T = U_k L_k^2 U_k^T$ Where $A_k = U_k L_k V_k^T$
- Document-document similarity: $A_k^T A_k = V_k L_k^2 V_k^T$
- Term document similarity (as an element of the transformed – document matrix)
- Extended query capabilities transforming initial query q to q_n : $q_n = q^T U_k L_k^{-1}$
- Thus q_n can be regarded a line in matrix V_k

Patterns in SVD Space



An Improved Model of Semantic Similarity Based on Lexical Co-Occurrence
Rohde et al. 2005

Patterns in SVD Space



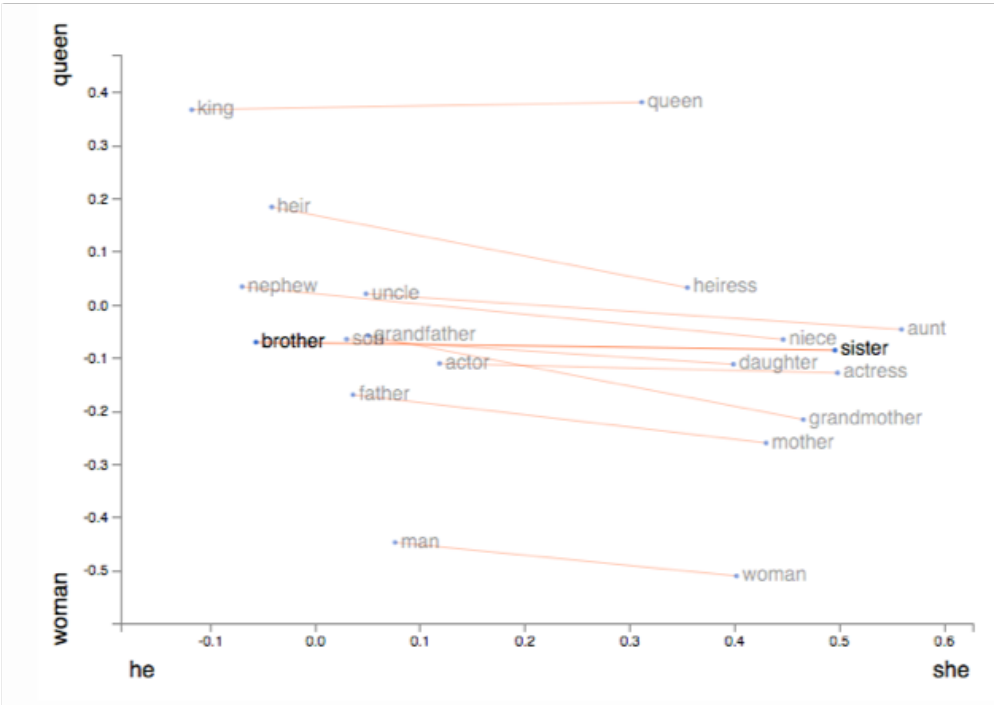
An Improved Model of Semantic Similarity Based on Lexical Co-Occurrence
Rohde et al. 2005

SVD problems

- The dimensions of the matrix change when dictionary changes
- The whole decomposition must be re-calculated when we add a word
- Sensitive to the imbalance in word frequency
- Very high dimensional matrix
- Not suitable for millions of words and documents
- Quadratic cost to perform SVD
- Solution: Directly calculate a low-dimensional representation

Word analogy

- Words with similar meaning laying close to each other
- Words sharing similar contexts may be analogous
  - Synonyms
  - Antonyms
  - Names
  - Colors
  - Places
  - Interchangeable words
- Vector computations with analogies
- i.e. **king - man + woman = queen**



https://lamyiowce.github.io/word2viz/

But why?

- what's an analogy?

$$\frac{p(w'|man)}{p(w'|woman} \approx \frac{p(w'|king)}{p(w'|queen)}$$

Assume PMI approximated by low rank approximation of the co-occurrence matrix.

1. $PMI(w', w) \approx v_w v_{w'}$ *inner product of the vector representations*
2. Isotropic: $E_{w'}[(v_{w'} v_u)]^2 = ||v_u||^2$
   Then,
3. $argmin_w E_{w'}[ln\frac{p(w'|w)}{p(w'|queen)} - ln\frac{p(w'|man)}{p(w'|woman)}]^2$
4. $argmin_w E_{w'}[(PMI(w'|w) - PMI(w'|queen) - PMI(w'|man) - PMI(w'|woman)]^2$
5. $argmin_w ||(v_w - v_{queen}) - (v_{man} - v_{woman})||^2$
6. $v_w \approx v_{queen} - v_{woman} + v_{man}$ which is an analogy!

- Arora et al (ACL 2016) shows that if (2) holds then (1) holds as well
- So we need to construct vectors from co-occurrence that satisfy (2)
- $d \ll |V|$ in order to have isotropic vectors

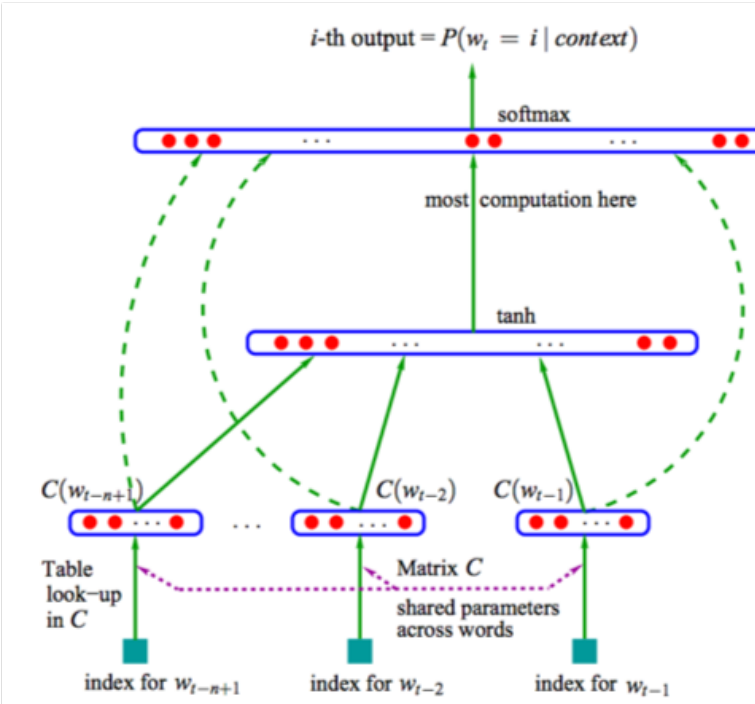
A Latent Variable Model Approach to PMI-based Word Embeddings, Arora et al (ACL 2016)

Learning Word Vectors

1. associate with each word in the vocabulary distributed *word feature* vector (a real-valued vector in $\mathbb{R}^m$)
2. express the joint *probability function* of word sequences in terms of the feature vectors of these words in the sequence, and
3. learn simultaneourly the *word feature vecotrs* and the parameters of that *probability function*

- Corpus containing T training words
- Objective :
  $f(w_t, \ldots, w_{t-n+1}) = \hat{P}(w_t | w_{t-n+1}, \ldots, w_{t-1})$
- Decomposed in two parts: $w_i \in V \xrightarrow{\text{mapping C}} \mathbb{R}^m$
  - Mapping **C** (1-hot v $\Rightarrow$ lower dimensions)
  - Mapping any **g** s.t. estimate prob $t+1|t$ previous)

  $f(w_{t-1}, \ldots, w_{t-n+1}) = g(C(w_{t-1}), \ldots, C(w_{t-n+1}))$
- C(i): i-th word feature vector(Word embedding)
- Objective function: $J = \frac{1}{T} \sum f(w_t, \ldots, w_{t-n+1})$

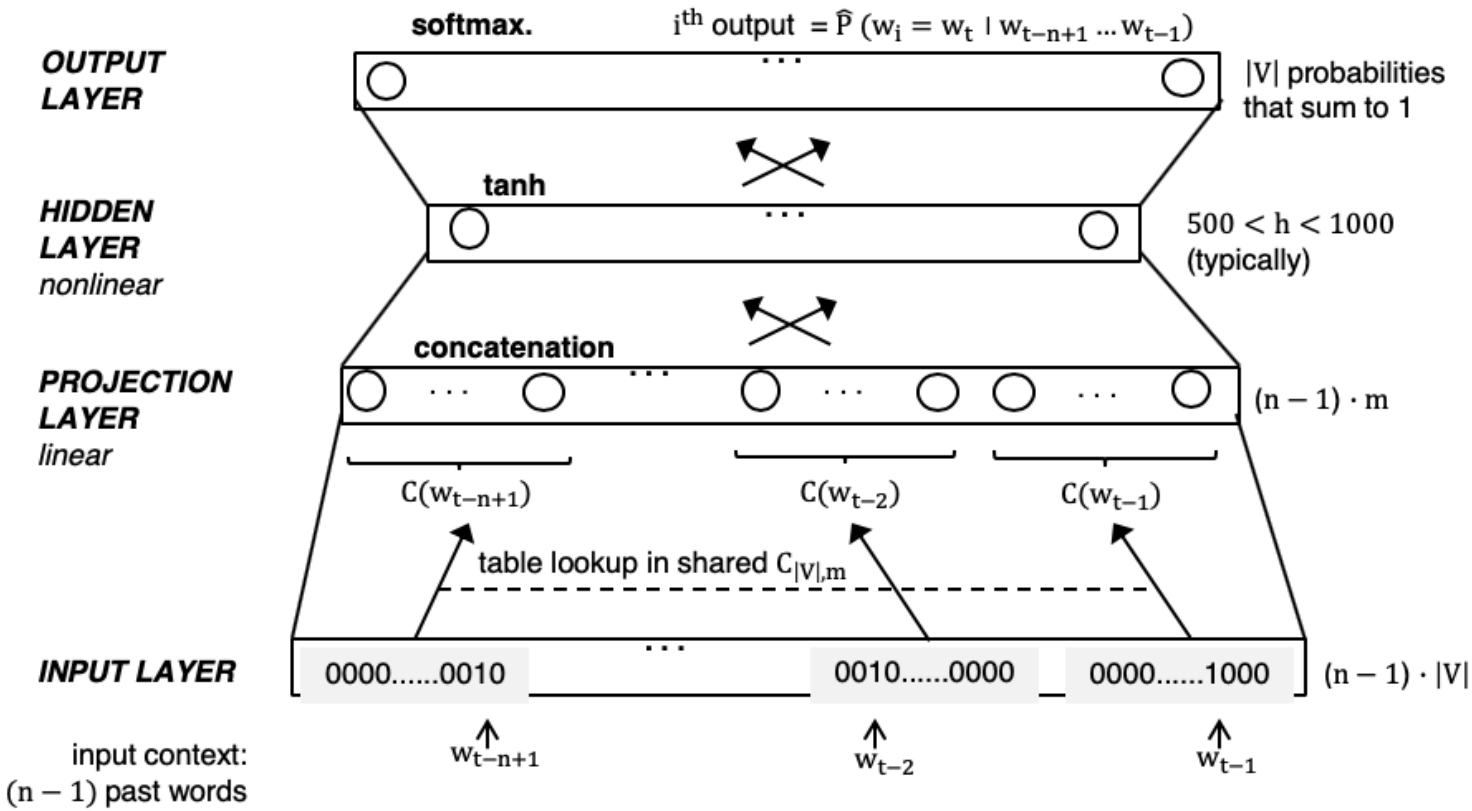


Bengio, Yoshua, et al. "A neural probabilistic language model." The Journal of Machine Learning Research 3 (2003): 1137-1155.

Neural Net Language Model

For each training sequence:

- input: (context, target)
- pair: $(w_{t-n+1}, \ldots, w_{t-1}, w_t)$
- objective: minimize $E = -log\hat{P}(w_t|w_{t-n+1}, \ldots, w_{t-1})$



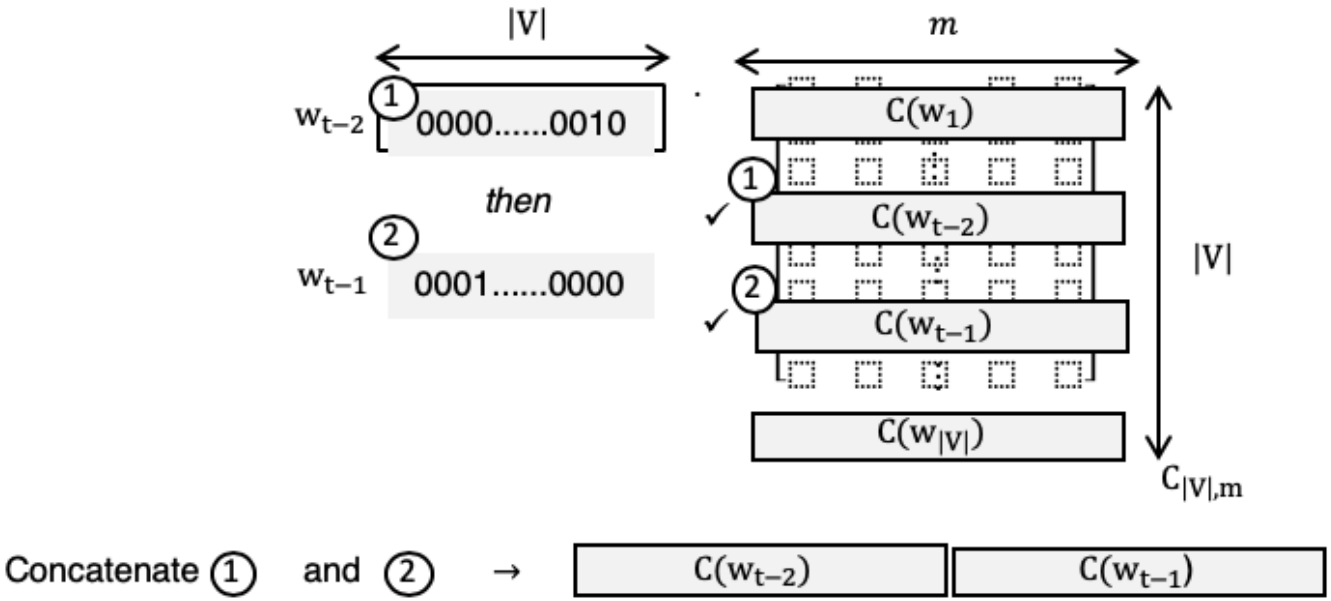
Objective function

- $E = -log\hat{P}(w_t|w_{t-n+1}, \ldots, w_{t-1})$
- a probability between 0 and 1.
- On this support, the log is negative $\Rightarrow$ –log term positive.
- makes sense to try to minimize it.
  - Probability of word given the context be as high as possible (1 for a perfect prediction).
  - case the error is equal to 0 (global minimum).

p	log(p)	-log(p)
0.7	-0.15490196	0.15490196
0.2	-0.698970004	0.698970004

NNLM Projection layer

- Performs a simple table lookup in $C_{|V|,m}$: concatenate the rows of the shared mapping matrix $C_{|V|,m}$ corresponding to the context words. Example for a two-word context w_{t-2} w_{t-1}:



- $C_{|V|,m}$ is **critical**: it contains the weights that are tuned at each step. After training, it contains what we're interested in: the **word vectors**

Bengio, Yoshua, et al. "A neural probabilistic language model." The Journal of Machine Learning Research 3 (2003): 1137-1155.

NNLM hidden/output layers and training

- Softmax (log-linear classification model) outputs positive numbers that sum to one (a multinomial probability distribution): i^{th} unit in the output layer:
  $\hat{P}(w_i = w_t | w_{t-n+1}, \ldots, w_{t-1}) = \frac{e^{yw_i}}{\sum_{i'=1}^{|V|} e^{yw_{i'}}}$

  where
  - $y = b + U \cdot tanh(d + H \cdot x)$
  - tanh: nonlinear squashing (link) function
  - x : concatenation C(w) of the context weight vectors seen previously
  - b : output layer biases ($|V|$ elements)
  - d : hidden layer biases (h elements). Typically $500 < h < 1000$
  - U: $|V| \times h$ matrix storing the hidden-to-output weights
  - H: $h \times (n-1)m$ matrix storing the projection-to-hidden weights

  $\boldsymbol{\theta = (b, d, U, H, C)}$
  complexity per training sequence: $n * m + n * m * h + \boldsymbol{h * |V|}$
  computational bottleneck: **nonlinear hidden layer** ($h * |V|$ term)

- Training performed via stochastic gradient descent (learning rate ϵ):

$$\theta \leftarrow \theta + \epsilon \frac{\partial E}{\partial \theta} = \theta + \epsilon \frac{\partial log \hat{P}(w_t | w_{t-n+1}, \ldots, w_{t-1})}{\partial \theta}$$

  (weights are initialized randomly, then updated via backpropagation)

NNLM facts

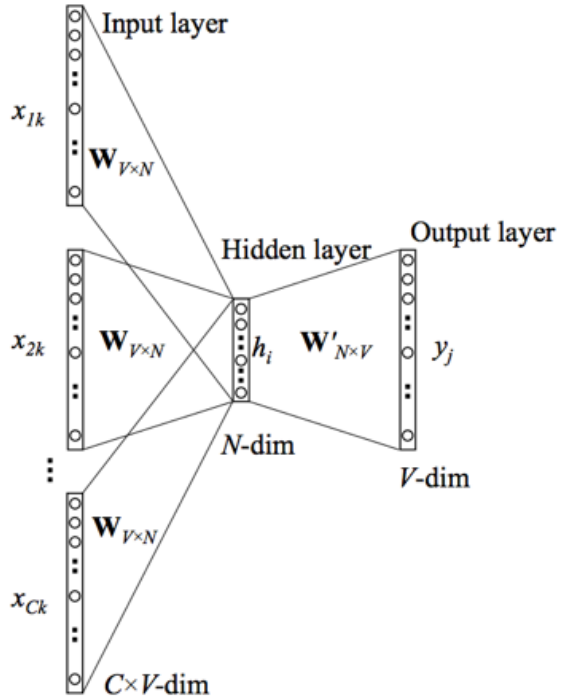
- tested on Brown (1.2M words, $|V| \cong 16K$) and AP News (14M words, $|V| \cong 150K$ reduced to 18K) corpuses
- h: # hidden units, n= # words in context, m: dimensionality of word vectors.
- Brown: h=100, n=5, m=30
- AP News: h=60, n=6, m=100, **3 week** training using **40 cores**
- 24% and 8% relative improvement (resp.) over traditional smoothed n-gram LMs
- in terms of *test set perplexity*: geometric average of $1/\hat{P}(w_t|w_{t-n+1}, \ldots, w_{t-1})$
- Due to **complexity**, NNLM can't be applied to large data sets $\rightarrow$ poor performance on rare words
- Bengio et al. (2003) claims main contribution was a more accurate LM. They let the interpretation and use of the word vectors as **future work**
- On the opposite, Mikolov et al. (2013) focus on the **word vectors**

Word2Vec

- Mikolov et al. in 2013
- Key idea of word2vec: achieve better performance not by using a more complex model (i.e., with more layers), but by allowing a **simpler (shallower) model** to be trained on **much larger amounts of data**
- no hidden layer (leads to 1000 $\times$ speedup)
- projection layer is shared (not just the weight matrix) - C
- context: words from both history & future:
- Two algorithms for learning words vectors:
  - **CBOW**: from context predict target
  - **Skip-gram**: from target predict context

CBOW

- continuous bag-of-words
- continuous representations whose order is of no importance
- uses the surrounding words to predict the center word
- n-words before and after the target word

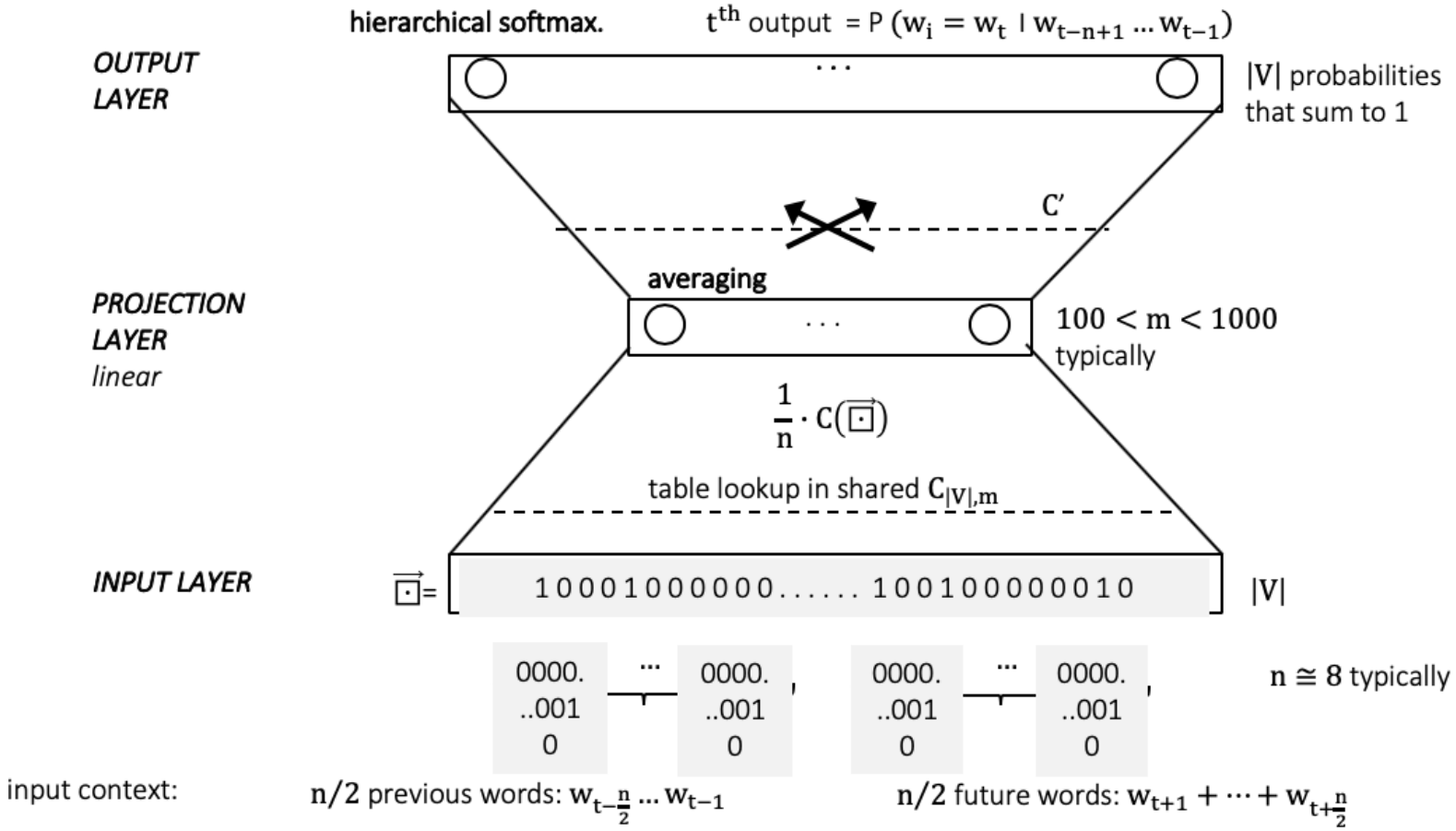


Efficient Estimation of Word Representations in Vector Space- Mikolov et al.

Continuous Bag-of-Words (CBOW)

For each training sequence:

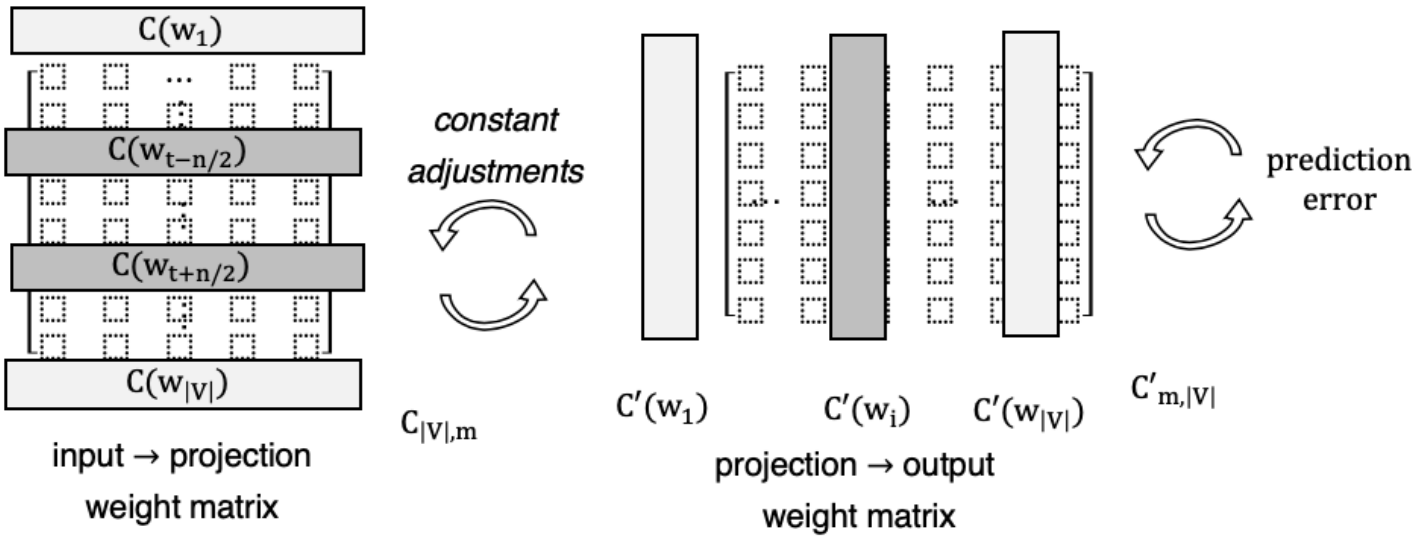
- input = (context, target)
- pair: $(w_{t-\frac{n}{2}}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+\frac{n}{2}}, w_t)$
- minimize $-log\hat{P}(w_t|w_{t-n+1}, \ldots, w_{t-1})$



Weight updating

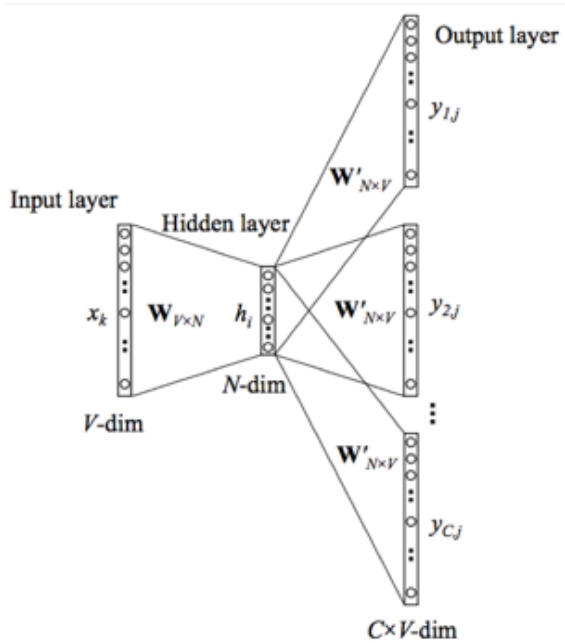
- For each (context, target=w_t) pair, only the word vectors from matrix C corresponding to the context words are updated
- Recall that we compute $P(w_i = w_t|context) \quad \forall w_i \in V$. We compare this distribution to the true probability distribution (1 for w_t, 0 elsewhere)
- **Back propagtion**
  - $P(w_i = w_t|context)$ is **overestimated** (i.e., > 0, happens in potentially $|V| - 1$ cases), some portion of $C'(w_i)$ is subtracted from the context word vectors in C, proportionally to the magnitude of the error
  - Reversely, if $P(w_i = w_t|context)$ **underestimated** (< 1, happens in potentially 1 case), some portion of $C'(w_i)$ is **added** to the context word vectors in C

  at each step the words move away or get closer to each other in the feature space $\rightarrow$ clustering



Skip-gram

- skip-gram uses, in a context, the center word to predict the surrounding words
- instead of computing the probability of the target word w_t given its previous words, we calculate the probability of the surrounding word w_{t+j} given w_t
- $P(w_{t+j}|w_t) = \frac{\exp(v_{w_t}^T v'_{w_{t+j}})}{\sum_{w \in V} \exp(v_{w_t}^T v'_{w_{t+j}})}$
- $v_{w_t}^T$ is a column of W_{VxN} and ${v'_{w_{t+j}}}^T$ is a column of W'_{NxV}
- objective function $J = \frac{1}{T} \sum_{t=1}^{T} \sum_{-n \leqslant j \leqslant n} logp(w_{t+j}|w_t)$



Efficient Estimation of Word Representations in Vector Space- Mikolov et al. 2013

word2vec facts

- Complexity is $\boldsymbol{n * m + m * log|V|}$ (Mikolov et al. 2013a)
- n:size of the context window (~ 10), $\boldsymbol{n \times m}$: dimensions of the projection layer, $\boldsymbol{|V|}$: size of the vocabulary
- On Google news 6B words training corpus, with $|V| \sim 10^6$:
  - CBOW with m=1000 took **2 days** to train on **140 cores**
  - Skip-gram with m=1000 took **2.5 days** on **125 cores**
  - NNLM (Bengio et al. 2003) took **14 days** on **180 cores**, for m=100 only! (note that m=1000 was not reasonably feasible on such a large training set)
- word2vec training *speed* $\cong 100K - 5M$ words/s
- Quality of the word vectors:
  - $\uparrow$ significantly with **amount of training data** and **dimension of the word vectors** (m), with diminishing relative improvements
  - measured in terms of accuracy on 20K semantic and syntactic association tasks. e.g., words in **bold** have to be returned:

Capital-Country	Past tense	Superlative	Male-Female	Opposite
Athens: **Greece**	walking: **walked**	easy: **easi-est**	brother: **sister**	ethical: **un-ethical**

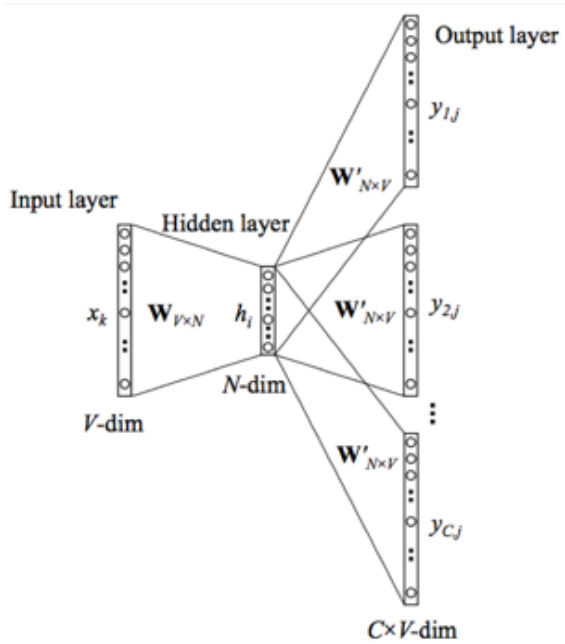
- Best NNLM: 12.3% overall accuracy. Word2vec (with Skip-gram): 53.3%

http://www.scribd.com/doc/285890694/NIPS-DeepLearningWorkshop-NNforText#scribd

https://code.google.com/p/word2vec/

Skip-gram

- skip-gram uses, in a context, the center word to predict the surrounding words
- instead of computing the probability of the target word w_t given its previous words, we calculate the probability of the surrounding word w_{t+j} given w_t
- $P(w_{t+j}|w_t) = \frac{\exp({v_{w_t}}^T v'_{w_{t+j}})}{\sum_{w \in V} \exp({v_{w_t}}^T v'_{w_{t+j}})}$
- ${v_{w_t}}^T$ is a column of $W_{V \times N}$ and $v'_{w_{t+j}}$ is a column of $W'_{N \times V}$
- objective function $J = \frac{1}{T} \sum_{t=1}^{T} \sum_{-n \leqslant j \leqslant n} \log p(w_{t+j}|w_t)$



Efficient Estimation of Word Representations in Vector Space- Mikolov et al. 2013

word2vec facts

- Complexity is $\boldsymbol{n * m + m * log|V|}$ (Mikolov et al. 2013a)
- n:size of the context window ($\sim$ 10), $\boldsymbol{n \times m}$: dimensions of the projection layer, $\boldsymbol{|V|}$: size of the vocabulary
- On Google news 6B words training corpus, with $|V| \sim 10^6$:
  - CBOW with m=1000 took **2 days** to train on **140 cores**
  - Skip-gram with m=1000 took **2.5 days** on **125 cores**
  - NNLM (Bengio et al. 2003) took **14 days** on **180 cores**, for m=100 only! (note that m=1000 was not reasonably feasible on such a large training set)
- word2vec training *speed* $\cong 100K - 5M$ words/s
- Quality of the word vectors:
  - $\uparrow$ significantly with **amount of training data** and **dimension of the word vectors** (m), with diminishing relative improvements
  - measured in terms of accuracy on 20K semantic and syntactic association tasks. e.g., words in **bold** have to be returned:

Capital-Country	Past tense	Superlative	Male-Female	Opposite
Athens: **Greece**	walking: **walked**	easy: **easiest**	brother: **sister**	ethical: **unethical**

- Best NNLM: 12.3% overall accuracy. Word2vec (with Skip-gram): 53.3%

---

http://www.scribd.com/doc/285890694/NIPS-DeepLearningWorkshop-NNforText#scribd

https://code.google.com/p/word2vec/

Glove

Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k\|ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k\|steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k\|ice)/P(k\|steam)$	8.9	8.5×10^{-2}	1.36	0.96

- Ratio of co-occurrence probabilities best distinguishes relevant words

  $F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}} \Rightarrow w_i^T \tilde{w}_k + b_i + \tilde{b}_k = log(X_{ik})$

- Cast this into a least square problem: $J = \sum_{i,j=1}^{V} f(X_{ij})(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - logX_{ij})^2$
- X: co-occurrence matrix
- f: weighting function $f(x) = \begin{cases} (x/x_{max})^{\alpha} & if \quad x < x_{max} \\ 1 & otherwise \end{cases}$
- b: bias terms
- w_i = word vector
- $\tilde{w}_j$ = context vector

model that utilizes

- count data
- bilinear prediction-based methods like word2vec

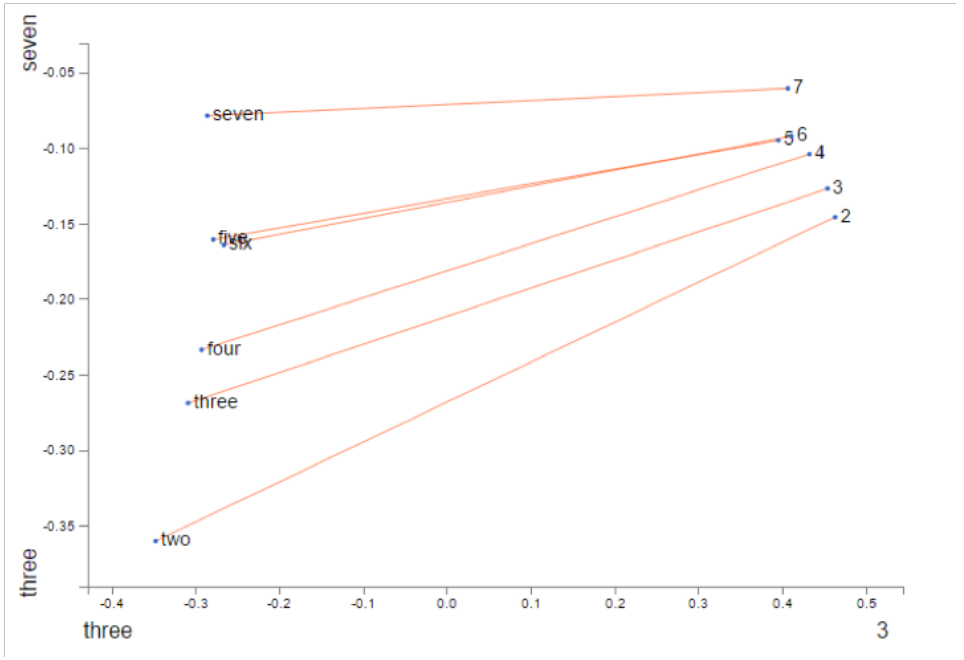
https://nlp.stanford.edu/pubs/glove.pdf

Which is better?

- Open question
- SVD vs word2vec vs GloVe
- All based on co-occurrence
- Levy, O., Goldberg, Y., & Dagan, I. (2015)
  - SVD performs best on similarity tasks
  - Word2vec performs best on analogy tasks
  - No single algorithm consistently outperforms the other methods
  - Hyperparameter tuning is important
  - 3 out of 6 cases, tuning hyperparameters is more beneficial than increasing corpus size
  - word2vec outperforms GloVe on all tasks
  - CBOW is worse than skip-gram on all tasks

Applications

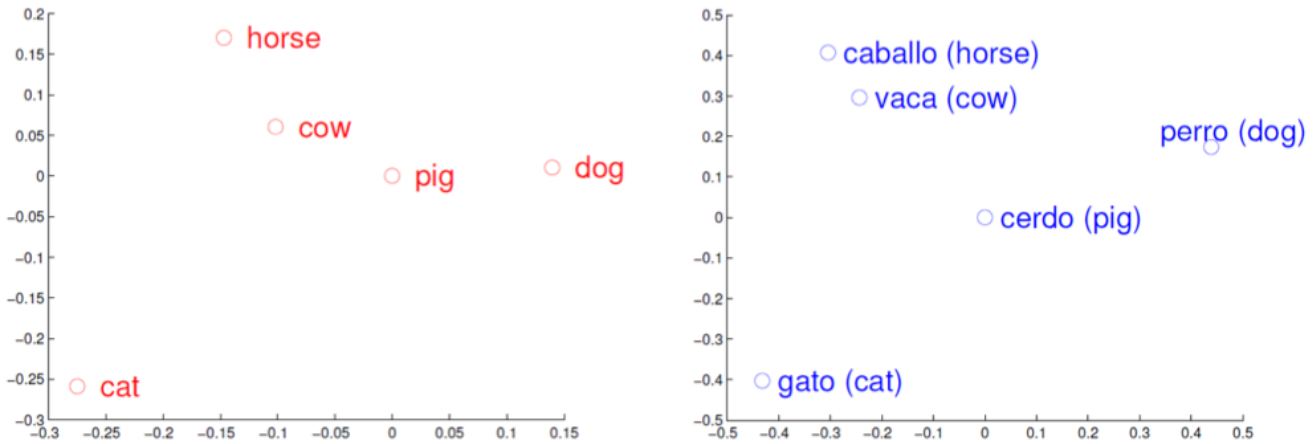
- Word analogies
- Find similar words
  - Semantic similarity
  - Syntactic similarity
- POS tagging
- Similar analogies for different languages
- Document classification



https://lamyiowce.github.io/word2viz/

Applications

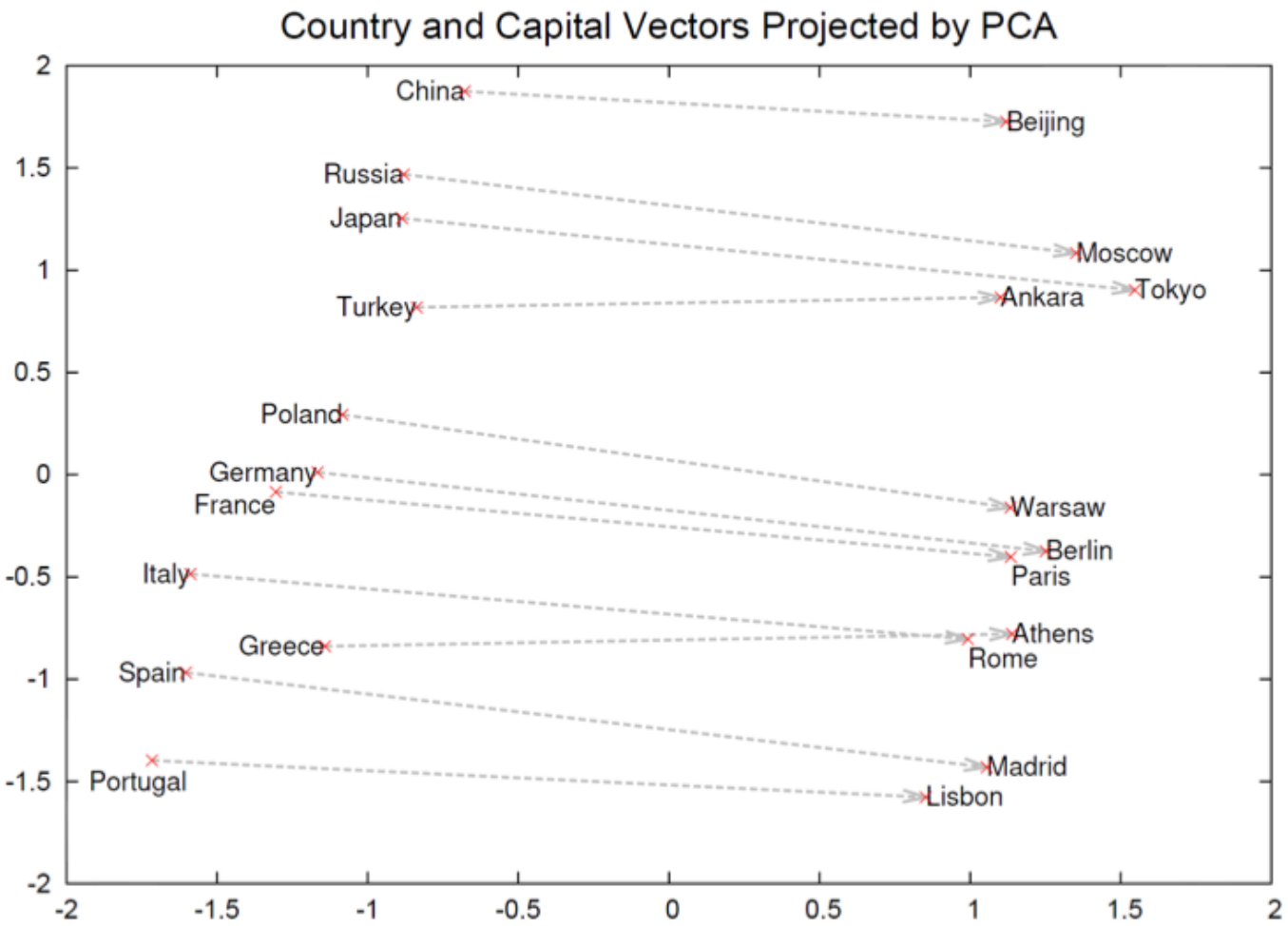
- High quality word vectors boost performance of all NLP tasks, including document classification, machine translation, information retrieval...
- Example for English to Spanish machine translation:



About 90% reported accuracy (Mikolov et al. 2013c)

Mikolov, T., Le, Q. V., & Sutskever, I. (2013). Exploiting similarities among languages for machine translation. arXiv preprint arXiv:1309.4168.

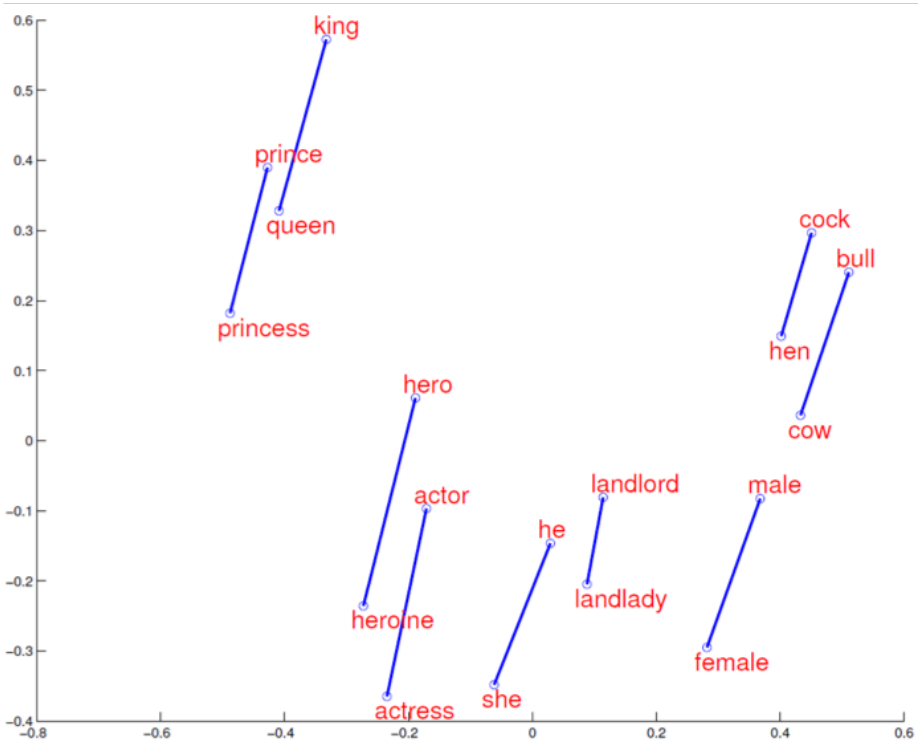
Remarkable properties of word vectors



regularities between words are encoded in the difference vectors e.g., there is a constant **country-capital** difference vector

Mikolov et al. (2013b) Distributed representations of words and phrases and their compositionality

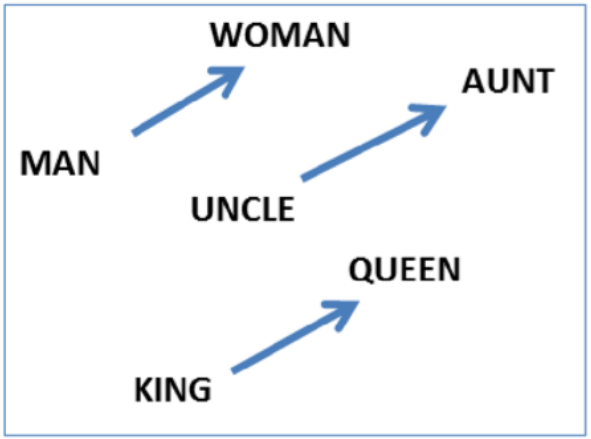
Remarkable properties of word vectors



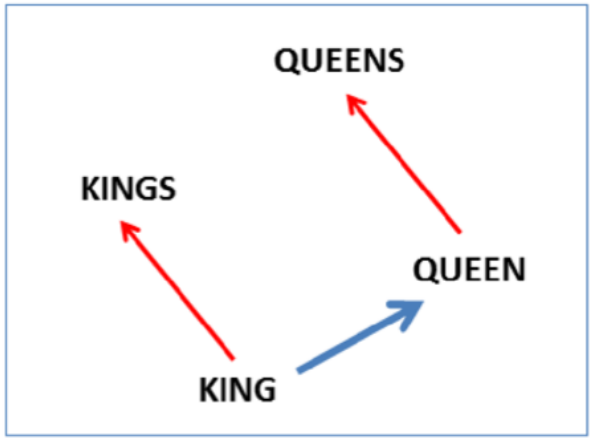
constant **female-male** difference vector

http://www.scribd.com/doc/285890694/NIPS-DeepLearningWorkshop-NNforText#scribd

Remarkable properties of word vectors



constant **female-male** difference vector



constant **singular-plural** difference vector

- Vector operations are supported and make intuitive sense:

$w_{king} - w_{man} + w_{woman} \cong w_{queen}$

$w_{paris} - w_{frace} + w_{italy} \cong w_{rome}$

$w_{windows} - w_{microsoft} + w_{google} \cong w_{android}$

$w_{einstein} - w_{scientist} + w_{painter} \cong w_{picasso}$

$w_{his} - w_{he} + w_{she} \cong w_{her}$

$w_{cu} - w_{copper} + w_{gold} \cong w_{au}$

- Online demo (scroll down to end of tutorial)

---

http://rare-technologies.com/word2vec-tutorial/

Outline

1. MLP and CNN applications to NLP
   - MLPs for Word embeddings
2. Recurrent NNs + LSTMs
3. Auto-encoders for unsupervised learning

CNN for text classification

- Use the high quality embeddings as input for Convolutional Neural Network
- Applies multiple filters to concatenated word vector

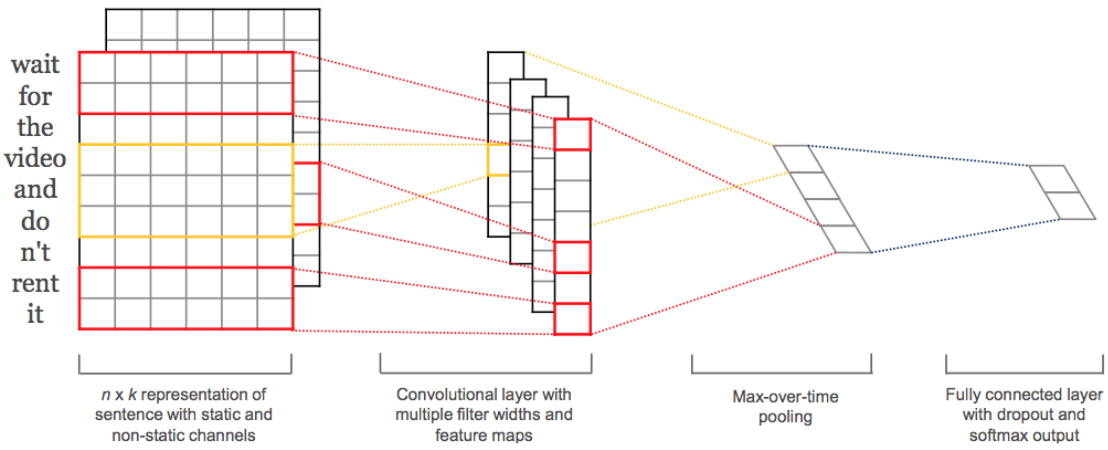
$$\mathbf{x}_{1:n} = \mathbf{x}_1 \oplus \mathbf{x}_2 \ldots \mathbf{x}_n$$

- Produces new features for every filter

$$c_i = f(\mathbf{w} \cdot \mathbf{x}_{i:i+h-1} + b)$$

- And picks the max as a feature for the CNN

$$\mathbf{c} = [c_1, c_2, \ldots, c_{n-n+1}] \quad \hat{c} = max\{\mathbf{c}\}$$



Yoon Kim - Convolutional Neural Networks for Sentence Classification

CNN for text classification

Many variations of the model [1]

- use existing vectors as input (CNN-static)
- learn vectors for the specific classification task through backpropagation (CNN-rand)
- Modify existing vectors for the specific task through backpropagation(CNN-non-static)

[1] Y. Kim, Convolutional Neural Networks for Sentence Classification, EMNLP 2014

CNN for text classification

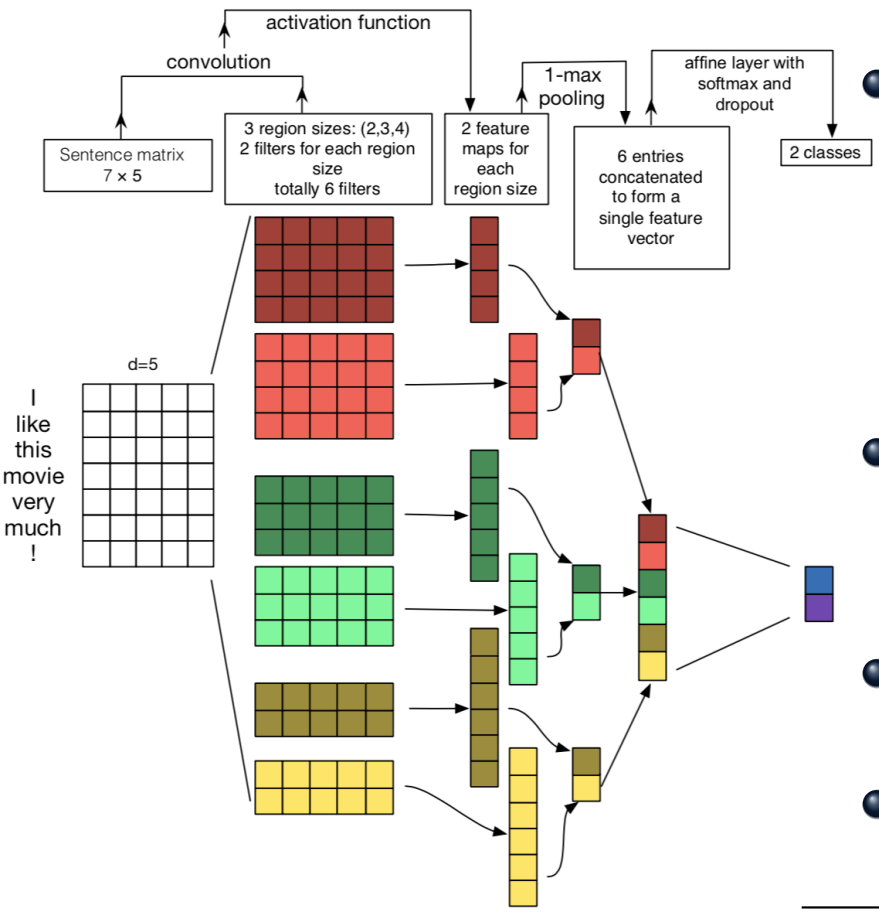
- Combine multiple word embeddings
- Each set of vectors is treated as a ‘channel’
- Filters applied to all channels
- Gradients are back-propagated only through one of the channels
- Fine-tunes one set of vectors while keeping the other static

CNN for text classification

Model	MR	SST-1	SST-2	Subj	TREC	CR	MPQA
CNN-rand	76.1	45.0	82.7	89.6	91.2	79.8	83.4
CNN-static	81.0	45.5	86.8	93.0	92.8	84.7	**89.6**
CNN-non-static	**81.5**	48.0	87.2	93.4	93.6	84.3	89.5
CNN-multichannel	81.1	47.4	**88.1**	93.2	92.2	**85.0**	89.4
RAE (Socher et al., 2011)	77.7	43.2	82.4	–	–	–	86.4
MV-RNN (Socher et al., 2012)	79.0	44.4	82.9	–	–	–	–
RNTN (Socher et al., 2013)	–	45.7	85.4	–	–	–	–
DCNN (Kalchbrenner et al., 2014)	–	48.5	86.8	–	93.0	–	–
Paragraph-Vec (Le and Mikolov, 2014)	–	**48.7**	87.8	–	–	–	–
CCAE (Hermann and Blunsom, 2013)	77.8	–	–	–	–	–	87.2
Sent-Parser (Dong et al., 2014)	79.5	–	–	–	–	–	86.3
NBSVM (Wang and Manning, 2012)	79.4	–	–	93.2	–	81.8	86.3
MNB (Wang and Manning, 2012)	79.0	–	–	**93.6**	–	80.0	86.3
G-Dropout (Wang and Manning, 2013)	79.0	–	–	93.4	–	82.1	86.1
F-Dropout (Wang and Manning, 2013)	79.1	–	–	**93.6**	–	81.9	86.3
Tree-CRF (Nakagawa et al., 2010)	77.3	–	–	–	–	81.4	86.1
CRF-PR (Yang and Cardie, 2014)	–	–	–	–	–	82.7	–
SVM_S (Silva et al., 2011)	–	–	–	–	**95.0**	–	–

Accuracy scores (Kim et al vs others)

CNN for text classification



- Data (text) only 1st column of input
- Rest of each row: embedding (in images 2D+RGB dimension)
- Filters of different sizes (4x5, 3x5 etc.)
  - Each size captures different features (need $\sim 10^2$ filters/size)
- Feature maps:
  - As many as the times filter fits on data matrix
- Max pooling maintains the “best features”
- Global feature map $\Rightarrow$ classification via softmax

---

[1] Zhang, Ye, and Byron Wallace. ”A sensitivity analysis of (and practitioners’ guide to) conv classification.” arXiv preprint arXiv:1510.03820 (2015).

CNN architecture for (short) document classification – T-SNE visualization

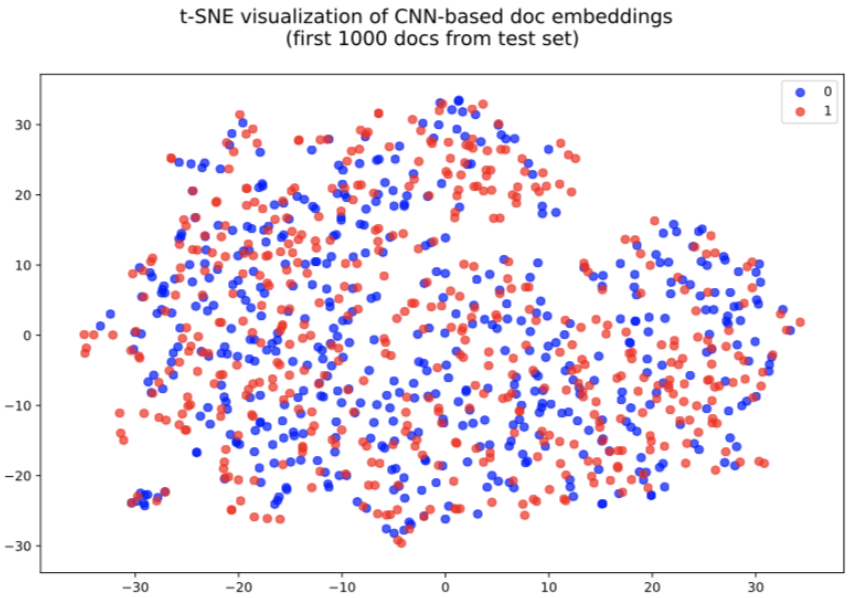


Figure 2: Doc embeddings before training.

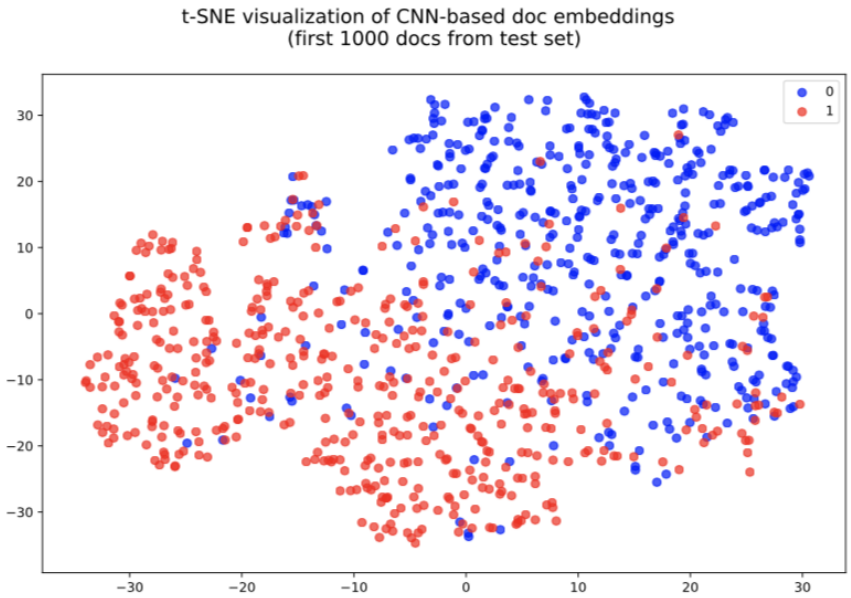


Figure 3: Doc embeddings after 2 epochs.

CNN architecture for (short) document classification - Saliency maps

- Words most related to changing the doc classification
- A in R^{sxd}, s:# sentence words, d:size of embeddings

$$saliency(a) = \left| \frac{\partial(CNN)}{\partial a} | a \right|$$

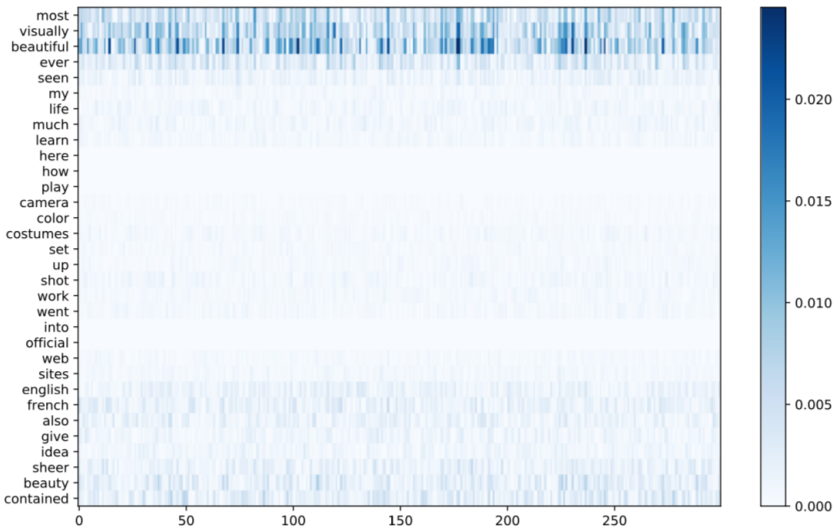


Figure 4: Saliency map for document 1 of the IMDB test set (true label: positive)

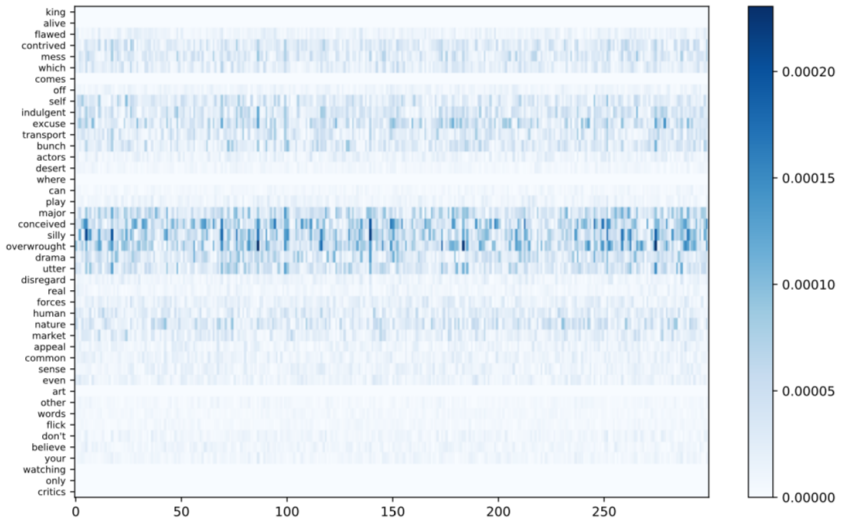
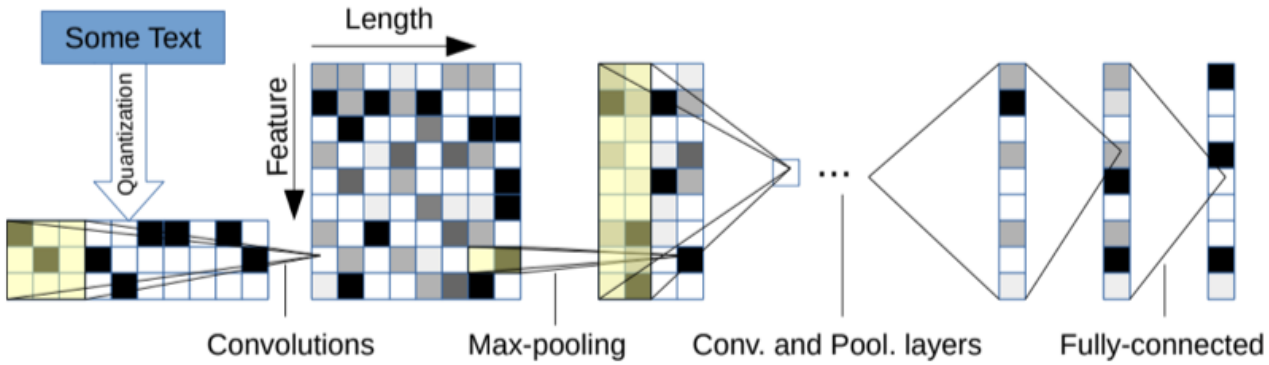


Figure 5: Saliency map for document 15 of the IMDB test set (true label: negative)

Character-level CNN for Text Classification

- Input: sequence of encoded characters
- quantize each character using “one-hot” encoding
- input feature length is 1014 characters
- 1014 characters able capture most of the texts of interest
- Also perform Data Augmentation using Thesaurus as preprocessing step

Model Architecture



- 9 layers deep
- 6 convolutional layers
- 3 fully-connected layers
- 2 dropout modules in between the fully-connected layers for regularization

Model Comparison

Model	AG	Sogou	DBP.	Yelp P.	Yelp F.	Yah. A.	Amz. F.	Amz. P.
BoW	11.19	7.15	3.39	7.76	42.01	31.11	45.36	9.60
BoW TFIDF	10.36	6.55	2.63	6.34	40.14	28.96	44.74	9.00
ngrams	7.96	2.92	1.37	**4.36**	43.74	31.53	45.73	7.98
ngrams TFIDF	**7.64**	**2.81**	**1.31**	4.56	45.20	31.49	47.56	8.46
Bag-of-means	**16.91**	**10.79**	**9.55**	**12.67**	**47.46**	**39.45**	**55.87**	**18.39**
LSTM	13.94	4.82	1.45	5.26	41.83	29.16	40.57	6.10
Lg. w2v Conv.	9.92	4.39	1.42	4.60	40.16	31.97	44.40	5.88
Sm. w2v Conv.	11.35	4.54	1.71	5.56	42.13	31.50	42.59	6.00
Lg. w2v Conv. Th.	9.91	-	1.37	4.63	39.58	31.23	43.75	5.80
Sm. w2v Conv. Th.	10.88	-	1.53	5.36	41.09	29.86	42.50	5.63
Lg. Lk. Conv.	8.55	4.95	1.72	4.89	40.52	29.06	45.95	5.84
Sm. Lk. Conv.	10.87	4.93	1.85	5.54	41.41	30.02	43.66	5.85
Lg. Lk. Conv. Th.	8.93	-	1.58	5.03	40.52	28.84	42.39	5.52
Sm. Lk. Conv. Th.	9.12	-	1.77	5.37	41.17	28.92	43.19	5.51
Lg. Full Conv.	9.85	8.80	1.66	5.25	38.40	29.90	40.89	5.78
Sm. Full Conv.	11.59	8.95	1.89	5.67	38.82	30.01	40.88	5.78
Lg. Full Conv. Th.	9.51	-	1.55	4.88	38.04	29.58	40.54	5.51
Sm. Full Conv. Th.	10.89	-	1.69	5.42	**37.95**	29.90	40.53	5.66
Lg. Conv.	12.82	4.88	1.73	5.89	39.62	29.55	41.31	5.51
Sm. Conv.	15.65	8.65	1.98	6.53	40.84	29.84	40.53	5.50
Lg. Conv. Th.	13.39	-	1.60	5.82	39.30	**28.80**	40.45	**4.93**
Sm. Conv. Th.	14.80	-	1.85	6.49	40.16	29.84	**40.43**	5.67

Testing errors for all models: Blue→best, Red→worst

Outline

1. MLP and CNN applications to NLP
   - MLPs for Word embeddings
2. Recurrent NNs + LSTMs
3. Auto-encoders for unsupervised learning

Recurrent Neural Networks

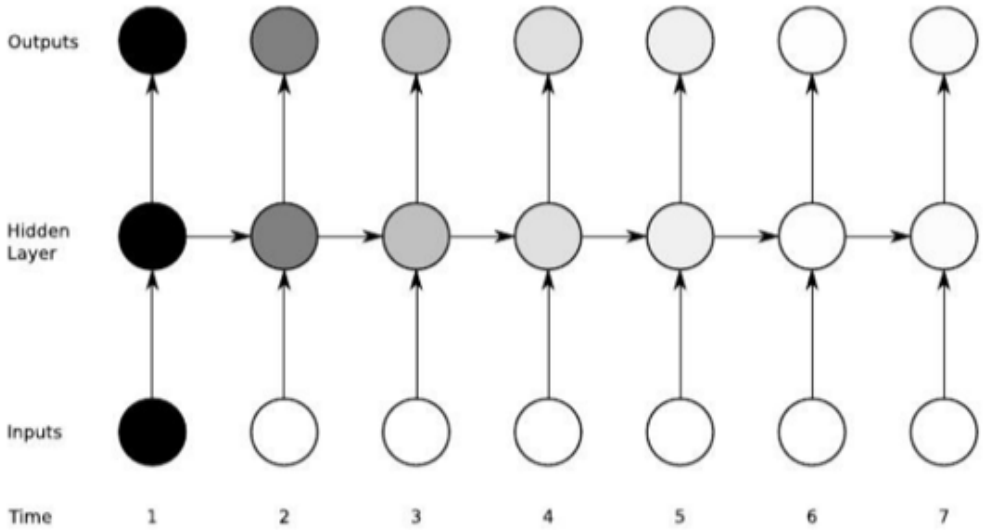
- CNNs are good at image classification
  - Input: an image
  - Output: probability of the class
    - $p(\textit{Red Panda}|\textit{image}) = 0.9$
    - $p(\textit{Cat}|\textit{image}) = 0.1$



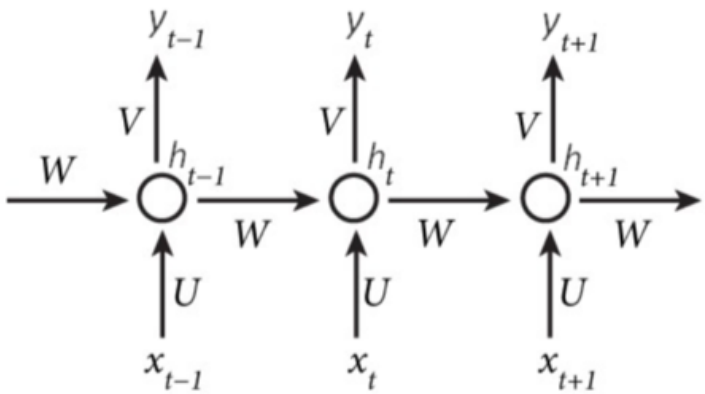
- Sequence learning: study of machine learning algorithms designed for sequential data (time series, language...)
  - Translate: ”machine learning is a challenging topic” to French: ”L’apprentissage automatique est un sujet difficile”
  - Predict the next word: ”John visited Paris, the capital of... ”
  - Input and output strongly correlated within the sequence.

Recurrent Neural Networks

- Update the hidden state in a deterministic nonlinear way.
- RNNs are powerful,
- Distributed hidden state that allows them to store a lot of information about the past efficiently.
- Non-linear dynamics that allows them to update their hidden state in complicated ways.
- No need to infer hidden state, pure deterministic.
- Weight sharing

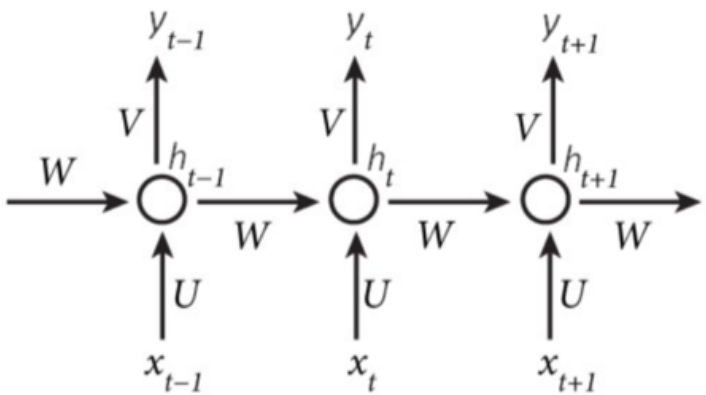


Recurrent Neural Networks



- input: ordered list of input vectors $x_1, \ldots, x_T$ initial hidden state h_0 initialized to all zeros
- output
  - ordered list of hidden states $h_1, \ldots, h_T$
  - ordered list of output vectors $y_1, \ldots, y_T$
  - The output vectors may serve as input for other RNN units, when considering deep architectures
  - The hidden states correspond to the "short-term" memory of the network.
- The last hidden state represents the encoding (embedding) of the time series

Recurrent Neural Networks



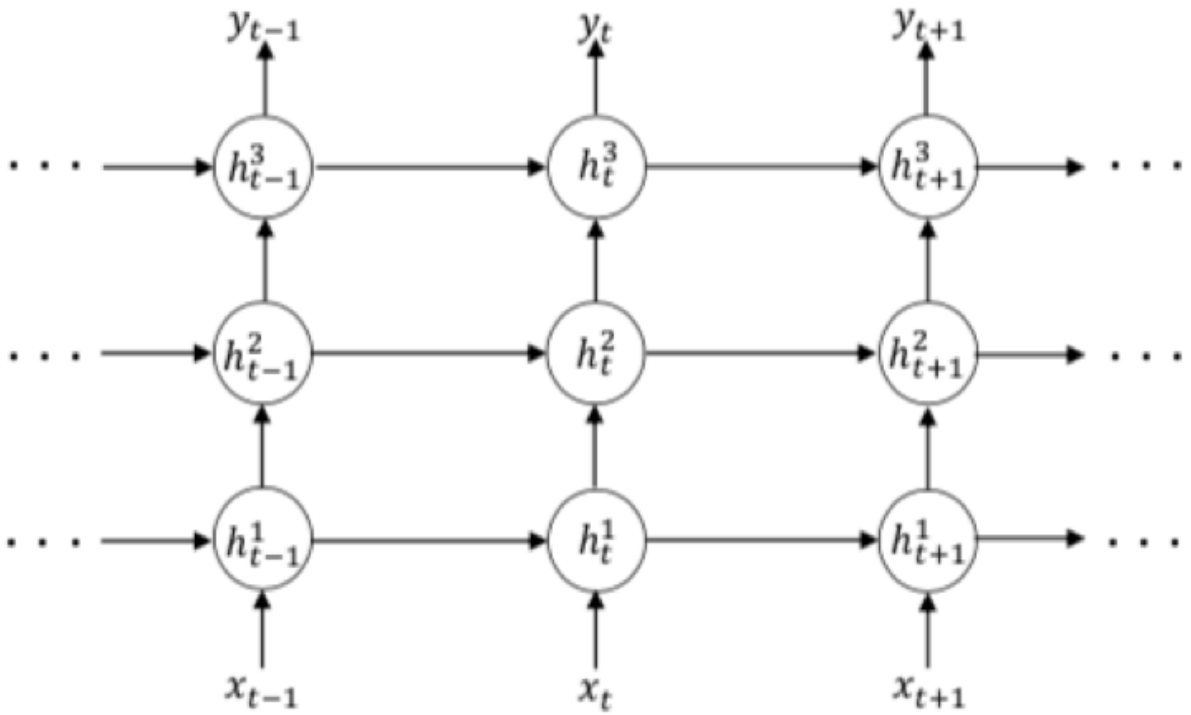
$$h_t = f(Ux_t + Wh_{t-1} + b)$$

- f: a nonlinear function
- $x_t \in \mathbb{R}^{d_{in}}$, $U \in \mathbb{R}^{H \times d_{in}}$, $W \in \mathbb{R}^{H \times H}$,

Parameter matrices

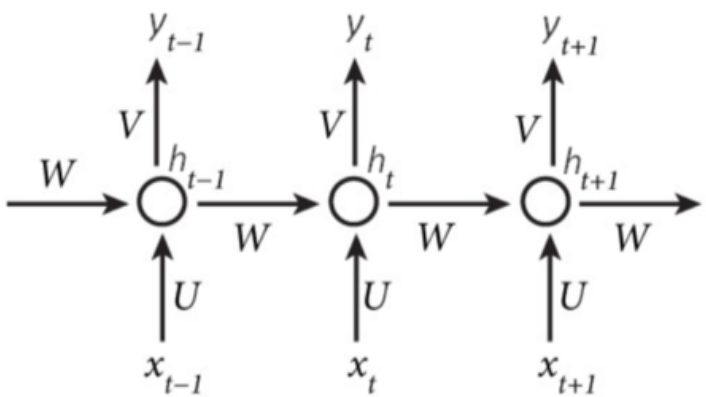
- d_{in}: size of the vocabulary
- H: dimension of the hidden layer ($H \sim 100$)
- $y_t \in \mathbb{R}^{d_{out}}$: transforms the current hidden state h_t to depend on the final task: i.e. for classification $y_t = softmax(Vh_t)$
- $V \in \mathbb{R}^{d_{out} \times H}$: parameter matrix shared across all steps (i.e. for word level language model $d_{out} = |V|$)

Deep RNNs



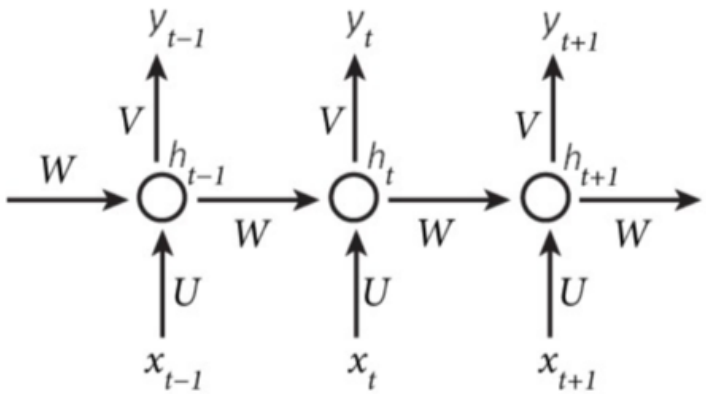
3 steps of an unrooled deep RNN. Each circle represents a RNN unit. The hidden state of each unit in the inner layer (1 & 2) serves as input to the corresponding unit in the layer above.

Recurrent Neural Networks



- CNNs are naturally efficient with $grids_8$,
- RNNs were specifically developed to be used with sequences
  - time series, or, in NLP, words (sequences of letters) or sentences (sequences of words).
  - language modeling $P[w_n|w_1, \ldots, w_{n-1}]$.
  - RNNs trained with such objectives can be used to generate new and quite convincing sentences from scratch
  - RNN can be considered as a chain of simple neural layers that share the same parameters

Learning in RNNs



- Assuming input
  $x_1, \ldots, x_{t-1}, x_t, x_{t+1}, \ldots, x_T$
- As a single time:

$$h_t = \sigma(Wh_{t-1} + Ux_t)$$

$$y_t = softmax(Vh_t)$$

Learning in RNNs

$$h_t = \sigma(Wh_{t-1} + Ux_t)$$
$$y_t = softmax(Vh_t)$$

- Main idea: we use the same set of W,U,V weights at all time steps!
- $h_0 \in \mathbb{R}^{H \times H}$ initialization vector for the hidden layer at time step 0
- $\hat{y} \in \mathbb{R}^{|V|}$ is a probability distribution over the vocabulary
- Loss function (@ time t): cross entropy predicting words instead of classes

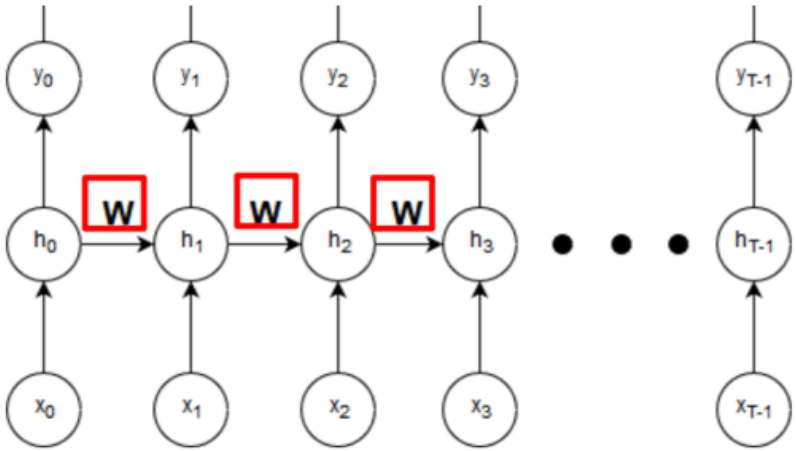
$$J^{(t)}(\theta) = -\sum_{j=1}^{|V|} y_{t,j} log \hat{y}_{t,j}$$

Learning in RNNs -backpropagation in time

- learning the weigths of **W**:

$$\mathbf{W} \rightarrow \mathbf{W} - \alpha \frac{\partial \mathbf{y}}{\partial \mathbf{W}}$$

- Computation involves summation over all paths regarding
  - i. itme $\frac{\partial \mathbf{y}}{\partial \mathbf{W}} = \sum_{j=0}^{T-1} \frac{\partial \mathbf{y}_j}{\partial \mathbf{W}}$
  - ii. Levels of hidden layers $\frac{\partial y_j}{\partial \mathbf{W}} = \sum_{k=1}^{j} \frac{\partial y_j}{\partial h_k} \frac{\partial h_k}{\partial \mathbf{W}}$



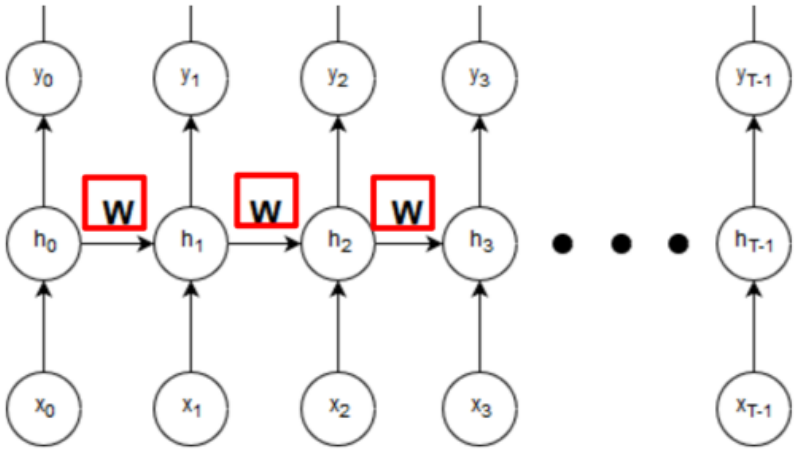
Learning in RNNs - backpropagation in time

- Therefore:

$$\frac{\partial y_j}{\partial \mathbf{W}} = \sum_{k=1}^{j} \frac{\partial y_j}{\partial h_j} \frac{\partial h_j}{\partial h_k} \frac{\partial h_k}{\partial \mathbf{W}}$$

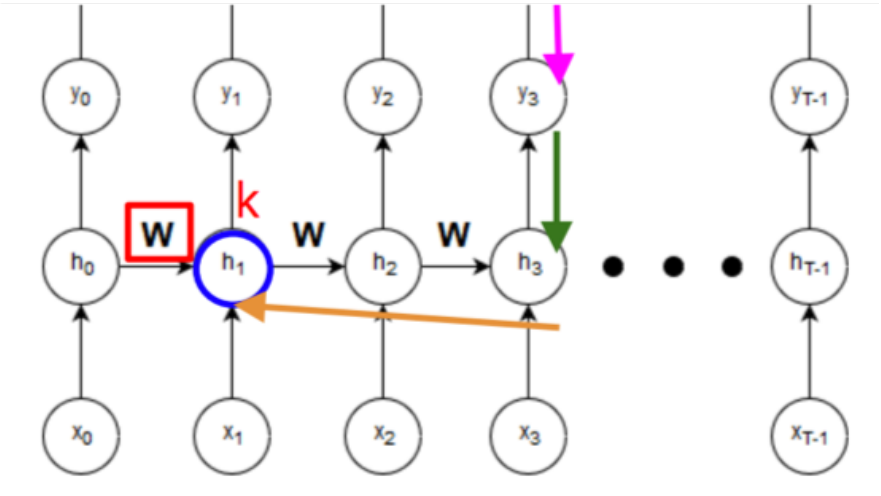
- Indirect dependency. One final use of the chain rule:

$$\frac{\partial h_j}{\partial h_k} = \prod_{m=k+1}^{j} \frac{\partial h_m}{\partial h_{m-1}}$$



Learning in RNNs - back propagation in time

$$\frac{\partial y_j}{\partial \mathbf{W}} = \sum_{j=0}^{T-1} \sum_{k=1}^{j} \frac{\partial y_j}{\partial h_j} \left(\prod_{m=k+1}^{j} \frac{\partial h_m}{\partial h_{m-1}} \right) \frac{\partial h_k}{\partial \mathbf{W}}$$



Learning in RNNs - vanishing/exploding gradients

$$\frac{\partial y_j}{\partial \mathbf{W}} = \sum_{j=0}^{T-1} \sum_{k=1}^{j} \frac{\partial y_j}{\partial h_j} \left(\prod_{m=k+1}^{j} \frac{\partial h_m}{\partial h_{m-1}} \right) \frac{\partial h_k}{\partial \mathbf{W}}$$

$$h_m = f(\mathbf{W}_h h_{m-1} + \mathbf{W}_x x_m)$$

$$\frac{\partial h_m}{\partial h_{m-1}} = \mathbf{W}_h^T \, diag(f'(\mathbf{W}_h h_{m-1} + \mathbf{W}_x x_m))$$

$$\frac{\partial h_j}{\partial h_k} = \prod_{m=k+1}^{j} \mathbf{W}_h^T \operatorname{diag}\left(f'(\mathbf{W}_h h_{m-1} + \mathbf{W}_x x_m)\right)$$

Weight Matrix

Derivative of activation function

Repeated matrix multiplications leads to vanishing and exploding gradients

Learning in RNNs - vanishing/exploding gradients

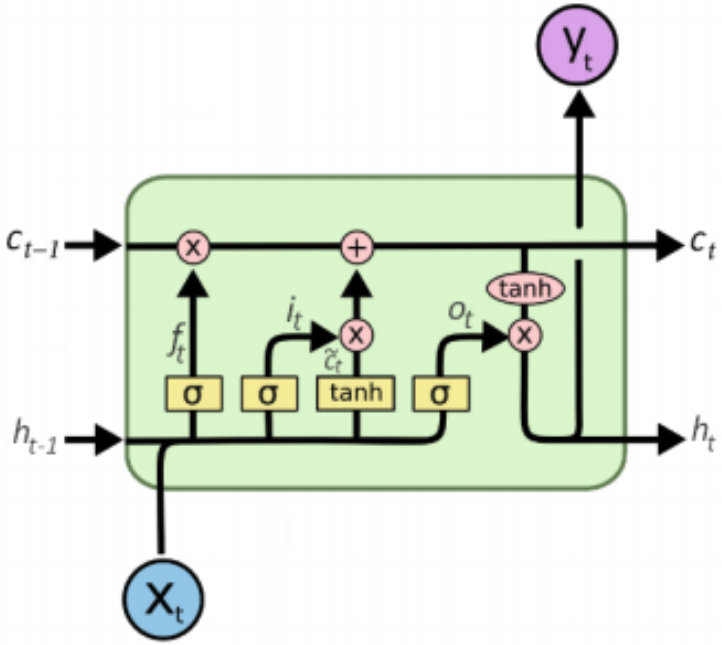
- Initialization of W with 1s
- Using relu as activation function f(z)= rect(z) = max(z, 0)
- Using - clip gradients to a maximum value [Mikolov]

Algorithm 1: Pseudo-code for norm clipping the gradients whenever they explode

---

$\hat{\mathbf{g}} \leftarrow \frac{\partial \xi}{\partial \theta}$;
if $||\hat{\boldsymbol{g}}|| \geq$ *threshold* **then**
  $\hat{\mathbf{g}} \leftarrow \frac{\textit{threshold}}{||\hat{\mathbf{g}}||}\hat{\mathbf{g}}$;
end

LSTM units

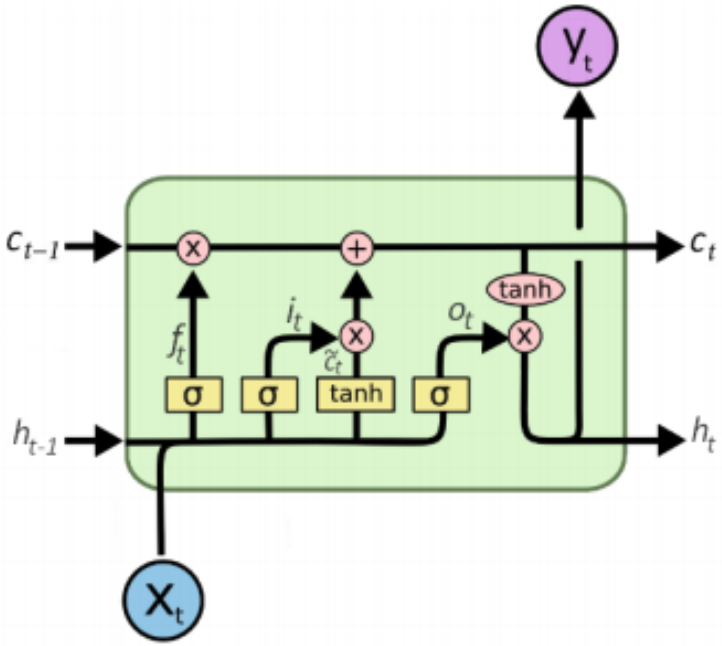


The LSTM unit. Adapted from Chris Colah's blog

1. forget gate layer: $f_t = \sigma(U_f x_t + W_f h_{t-1} + b_f)$
2. input gate layer: $i_t = \sigma(U_i x_t + W_i h_{t-1} + b_i)$
3. candidate values computation layer: $\tilde{c}_t = tanh(U_c x_t + W_c h_{t-1} + b_c)$
4. output gate layer: $o_t = \sigma(U_o x_t + W_o h_{t-1} + b_o)$

- To tackle RNN problems of i. vanishing gradients and ii. Keep track of information for longer term.
- **Cell state** (C_t) "memory" of the network; carries information across time steps.
- **Hidden state** (h_t) short-term output passed to next LSTM unit.
- **Gates** — control how information flows:
  - **Forget gate:** what to erase from memory.
  - **Input gate:** new information to store.
  - **Output gate:** output at the current step.

LSTM units



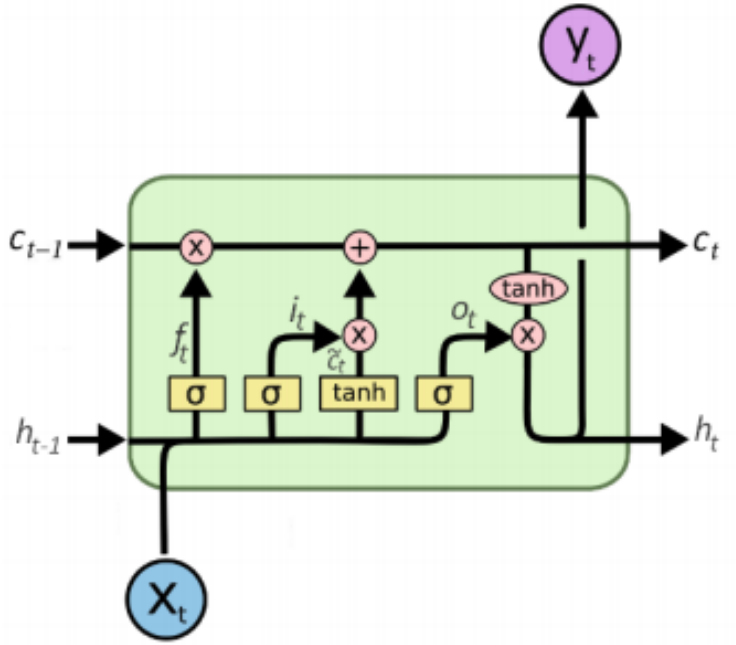
The LSTM unit. Adapted from Chris Colah's blog

1. forget gate layer: $f_t = \sigma(U_f x_t + W_f h_{t-1} + b_f)$
2. input gate layer: $i_t = \sigma(U_i x_t + W_i h_{t-1} + b_i)$
3. candidate values computation layer: $\tilde{c}_t = tanh(U_c x_t + W_c h_{t-1} + b_c)$
4. output gate layer: $o_t = \sigma(U_o x_t + W_o h_{t-1} + b_o)$

Assume a new training example x_t and the current hidden state h_{t-1},

- forget gate layer ft determines how much of the previous cell state c_{t-1} should be forgotten (what fraction of the memory should be freed up),
- input gate layer decides how much of the candidate values $\tilde{c}_t$ should be written to the memory: how much of the new information should be learned. Combining the output of the two filters updates the cell state:
  - $c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t$
  - $h_t = tanh(c_t) \circ o_t$
  - $y_t = softmax(Vh_t)$

LSTM units



The LSTM unit. Adapted from Chris Colah's blog

1. forget gate layer: $f_t = \sigma(U_f x_t + W_f h_{t-1} + b_f)$
2. input gate layer: $i_t = \sigma(U_i x_t + W_i h_{t-1} + b_i)$
3. candidate values computation layer: $\tilde{c}_t = tanh(U_c x_t + W_c h_{t-1} + b_c)$
4. output gate layer: $o_t = \sigma(U_o x_t + W_o h_{t-1} + b_o)$

Example: Predicting the Next Word
Assume train an LSTM on the sentence:

"The weather is very ___"

LSTM processes one word at a time:

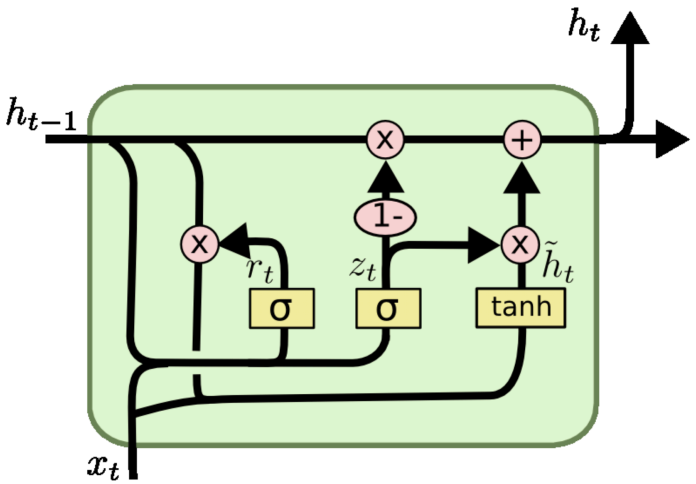
$x_1 =$ "The", $x_2 =$ "weather",

$x_3 =$ "is", $x_4 =$ "very"
At each step, LSTM updates its internal memory to remember the context ("talking about weather"). When it reaches the blank, it predicts the next word — "hot," "cold," or "rainy."
If trained well, LSTM will output a probability distribution like:

$P($"hot"$) = 0.6, P($"cold"$) = 0.3,$

P("rainy")=0.1

GRU units



The GRU unit

Gated Recurrent Unit (GRU): simplified version of LSTM network, [Cho et al.-2014)]. RNN architecture designed to handle vanishing gradient and capture long-term dependencies — but with *fewer parameters and simpler structure than LSTMs.*

GRU combines cell state and hidden state of an LSTM into a single vector h_t anduses only two gates instead of three:

- **Update Gate** (z_t): controls how much of the past information to keep.
- **Reset Gate** (r_t): controls how much of past information to forget when computing the new candidate.

GRU Equations At each time step t, given:

- Input x_t and Previous hidden state h_{t-1}

The GRU updates as follows:

$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$

$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$

$_t = \tanh(W_h \cdot [r_t * h_{t-1}, x_t] + b_h)$

$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$

GRU vs LSTM units

Feature	LSTM	GRU
Number of gates	3 (input, forget, output)	2 (update, reset)
Cell state	Separate cell C_t	Combined with hidden h_t
Parameters	More (slower to train)	Fewer (faster to train)
Performance	Often slightly better on complex data	Comparable or better on smaller datasets

Outline

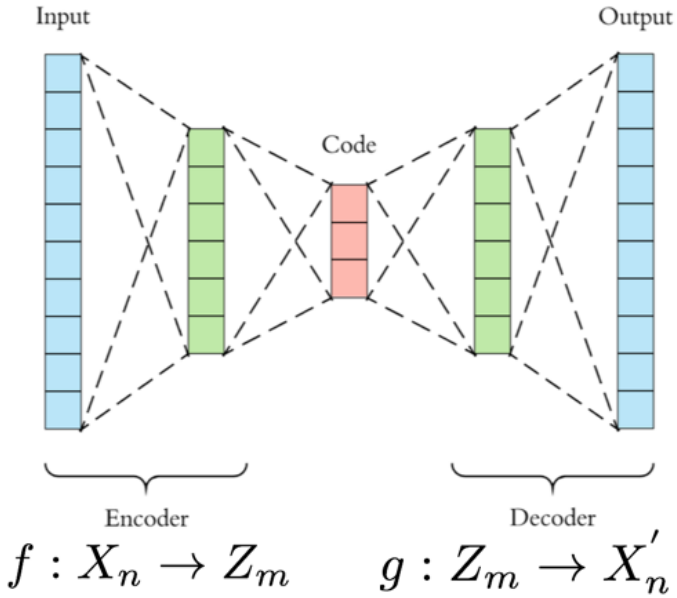
1. MLP and CNN applications to NLP
   - MLPs for Word embeddings
2. Recurrent NNs + LSTMs
3. Auto-encoders for unsupervised learning

Autoencoders - the concept

- introduced by Hinton
  - address the problem of “backpropagation without a teacher”
  - Need to formulate an error function – using input data as the teacher [RUM1986]
- more recently, “deep architecture” approach (Hinton et al., 2006; Hinton and Salakhutdinov, 2006; Bengio and LeCun, 2007;Erhan et al., 2010)
  - Restricted Boltzmann Machines (RBMS), stacked, trained bottom up in unsupervised fashion.

Autoencoders

- Assume a set of n-dim data X_n - we assume transformations f, g
  - f : mapping data to a lower dim space Z_m (code)
  - g: mapping Z_m to the an original dim data $X'_n \sim X_n$
- Learning: minimize the loss $L(x(f(g(x))$
  - $|code| < |input|$: undercomplete AE
  - Sparse autoencoders: $L(x(f(g(x)) + \Omega(Z_m)$



Applications of AEs

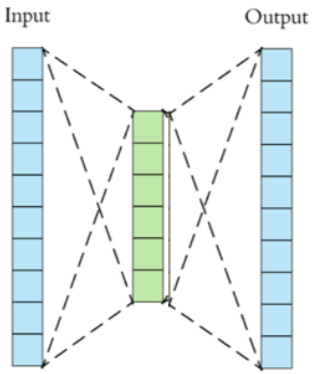
- Learn a low dimensional Z_m representation of the data X
  - learning low dimensional space with less reconstruction error than PCA (Hinton 2006)
  - Features used in classification and improve generalization (Hinton 2007)
- Perform clustering – unsupervised learning
- Information Retrieval
  - Semantic hashing for images and NLP (Hinton 2007, Torralba 2007)
- Learn data features in the absence of a supervised task

Sparse Autoencoders (AE)

- Assume AE: n input neurons, m neurons in hidden layer and k training data
- Assume $a_j^{(2)}(x^{(i)})$ activation of j-th neuron in hidden layer for x(i)-th data point
- average output of j-th neuron of the hidden layer after data training: $\hat{\rho} = \frac{1}{k}\sum_i^k (a_j^{(2)}(x^{(i)}))$
- Assume only few hidden layer neurons should fire for each training sample
  - output ρ of a neuron should be small (i.e. 0.02)
  - Penalize $\hat{\rho}$ deviating significantly from ρ:

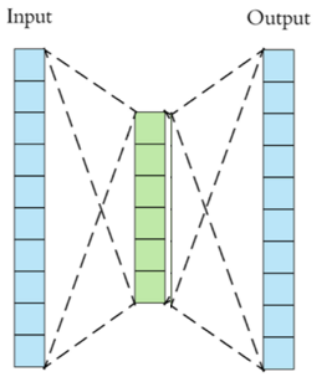
$$\sum_{j=1}^{m} \rho log \frac{\rho}{\hat{\rho}_j} + (1-\rho) log \frac{1-\rho}{1-\hat{\rho}_j} = \sum_{j=1}^{m} KL(\rho||\hat{\rho}_j)$$

  - KL divergence metric of distribution difference – sparsity term



Sparse Autoencoders (AE)

- The overall cost function:
  $J_{sparse}(W, b) = J(W, b) + \beta \sum_{j=1}^{m} KL(\rho||\hat{\rho}_j)$
- Need to incorporate the KL divergence in the back-propagation
- The back-prop at the second (hidden) layer:
  $\delta_i^{(2)} = (\sum_{j=1}^{m} W_{ji}^{(2)} \delta_j^{(3)}) f'(z_i^{(2)})$
- Take into account sparsity:
  $\delta_i^{(2)} = (\sum_{j=1}^{m} W_{ji}^{(2)} \delta_j^{(3)}) + \beta(-\frac{\rho}{\hat{\rho}_i} + \frac{1-\rho}{1-\hat{\rho}_i}) f'(z_i^{(2)}))$
- Images that enable hidden units are image fundamental features (i.e. basic shapes)



Auto-encoders for clustering [Hie te al 2016]

Deep Embedding Clustering (DEC)

- Assume data X pf n points, cluster them into k. clusters in a lower dimensional space via a nonlinear mapping $f_\theta : X \rightarrow Z$, Z, θ: code, trainable parameters
- learning simultaneously i. the center of the k clusters in the Z space and μ,θ
- DEC phases:
  - parameter initialization with a deep stacked autoencoder (Vincent 2010)
  - parameter optimization (i.e., clustering), where we iterate between computing an auxiliary target distribution and minimizing the Kullback–Leibler (KL) divergence to μ

Auto-encoders for clustering [Hie te al 2016]

- soft assignment of embedded point $z_i = f_\theta(x_i)$ and the cluster centroids μ_j

$$q_{ij} = \frac{(1 + ||z_i - \mu_j||^2/\alpha)^{-\frac{\alpha+1}{2}}}{\sum_{j'} (1 + ||z_i - \mu_{j'}||^2/\alpha)^{-\frac{\alpha+1}{2}}}$$

- iteratively refine clusters by learning from their high confidence assignments with the help of an auxiliary target distribution

$$p_{ij} = \frac{q_{ij}^2/f_j}{\sum_{j'} q_{ij'}^2/f_{j'}}$$

where $f_j = \sum_i q_{ij}$are the soft cluster frequencies.

- Then the error is $L = KL(P||Q) = \sum_i \sum_j p_{ij} log \frac{p_{ij}}{q_{ij}}$

Auto-encoders for clustering [Hie te al 2016]

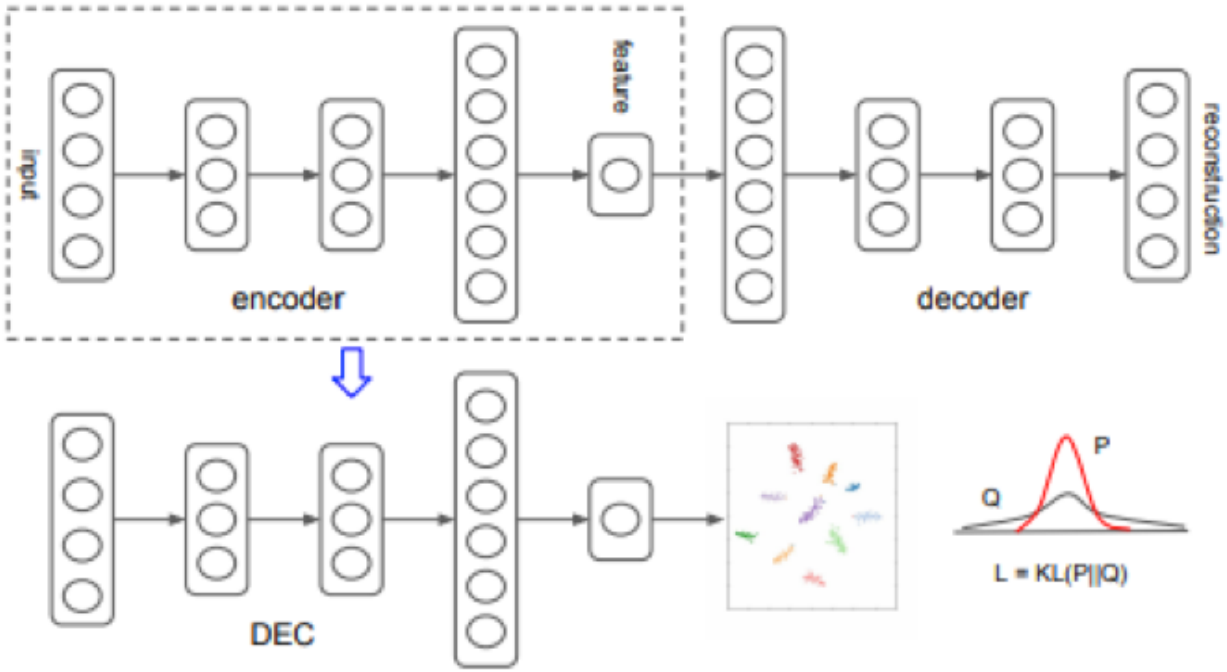
- jointly optimize cluster centers $\{\mu_j\}$ and DNN parameters θ using Stochastic Gradient Descent (SGD):

$$\frac{\partial L}{\partial z_i} = \frac{\alpha+1}{\alpha} \sum_j (1 + \frac{||z_i - \mu_j||^2}{\alpha})^{-1} \times (p_{ij} - q_{ij})(z_i - \mu_j)$$

$$\frac{\partial L}{\partial \mu_j} = -\frac{\alpha+1}{\alpha} \sum_i (1 + \frac{||z_i - \mu_j||^2}{\alpha})^{-1} \times (p_{ij} - q_{ij})(z_i - \mu_j)$$

- gradients $\frac{\partial L}{\partial z_i}$ are passed down to the DNN
- gstandard backpropagation compute DNN's parameter gradient $\frac{\partial L}{\partial \theta}$
- discovering cluster assignments, stop when less than tol % of points change cluster assignment between consecutive iterations.

Auto-encoders for clustering [Hie te al 2016]



To initialize the cluster centers:

- Pass data through the initialized DNN to get embedded data points and then perform standard k-means clustering in the feature space Z to obtain k initial centroids $\{\mu_j\}_{j=1}^{k}$.

Auto-encoders for clustering [Hie te al 2016]

Dataset	# Points	# classes	Dimension	% of largest class
MNIST (LeCun et al., 1998)	70000	10	784	11%
STL-10 (Coates et al., 2011)	13000	10	1428	10%
REUTERS-10K	10000	4	2000	43%
REUTERS (Lewis et al., 2004)	685071	4	2000	43%

Method	MNIST	STL-HOG	REUTERS-10k	REUTERS
k-means	53.49%	28.39%	52.42%	53.29%
LDMGI	84.09%	33.08%	43.84%	N/A
SEC	80.37%	30.75%	60.08%	N/A
DEC w/o backprop	79.82%	34.06%	70.05%	69.62%
DEC (ours)	**84.30%**	**35.90%**	**72.17%**	**75.63%**

Auto-encoders for clustering [Hie te al 2016]



(a) MNIST



(b) STL-10

Top 10 results for each of the 10 clusers for the MNIST and STL data sets

Why is Unsupervised learning important

- Machine learning: learning representations (features, latent variables) to capture the statistical dependencies
- Supervised learning: from input variables to output variables, – i.e. categories
  - supervised learning require ”teaching” computer concepts that matter to humans
  - supervised deep learning discover meaningful intermediate representations in their layers.
- Unsupervised learning
  - capture all possible dependencies between any subset of variables
  - human learning: based on observation and unsupervised interaction (action-perception loop)
- Issues
  - Training sets increasingly difficult to keep the pace with data production
  - Can we trust humans ?

“. . . hope is that deep unsupervised learning will be able to discover (possibly with a little bit of help from the few labeled examples we can provide) all of the concepts and underlying causes that matter (some being explicitly labeled, some remaining unnamed) to explain what we see around us. So I believe it is essential for approaching AI to make progress in this direction. And we are ;-)” Y. Bengio